POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa licencjacka

# DOKUMENTOWA BAZA DANYCH DLA DZIENNIKÓW ZDARZEŃ

Konrad Maciejewski, 151869
Tsimafei Kotski, 158740

Promotor
dr hab. inż. Tomasz Pawlak, prof. PP

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

# Spis treści


**1 Wstęp** **1**

1.1 Motywacja . . . 1

1.2 Cel i zakres pracy . . . 2

1.3 Podział pracy . . . 3

1.4 Struktura pracy . . . 3

**2 Problem interpretacji języka PQL** **4**

2.1 Standard eXtensible Event Stream . . . 4

2.2 Język PQL . . . 6

2.3 Problem interpretacji języka PQL . . . 7

**3 Przegląd literatury** **9**

3.1 Języki zapytań do dzienników zdarzeń . . . 9

3.2 Systemy baz danych dla dzienników zdarzeń . . . 10

**4 Interpreter PQL** **13**

4.1 Parser PQL . . . 13

4.2 Translator PQL do Mango Query . . . 15

4.3 Wykonywanie zapytań . . . 18

4.4 Sposoby zapewnienia wydajności . . . 23

**5 Eksperymenty** **26**

5.1 Pytania eksperymentalne . . . 26

5.2 Środowisko eksperymentalne . . . 27

5.3 Zgodność wyników ze specyfikacją PQL . . . 28

5.4 Niezmienność danych (Lossless Round-trip) . . . 29

5.5 Skalowalność względem rozmiaru danych wejściowych . . . 31

5.6 Narzut pamięciowy optymalizacji architektonicznych . . . 33

**6 Dyskusja** **36**

6.1 Zmiana paradygmatu: SQL vs. NoSQL w analizie procesów . . . 36

6.2 Przeniesienie logiki z bazy danych do warstwy aplikacji . . . 36

6.3 Zalety i wady podejścia dokumentowego . . . 38

6.4 Zastosowanie i perspektywy rozwoju . . . 39

**7 Wnioski i dalszy kierunek prac** **40**
7.1 Podsumowanie zrealizowanych prac . . . . . 40
7.2 Realizacja celu pracy i weryfikacja hipotezy . . . . . 40
7.3 Najważniejsze wnioski i osiągnięcia . . . . . 41
7.4 Dalszy kierunek prac . . . . . 41

**Literatura** **43**

# Rozdział 1

# Wstęp

## 1.1 Motywacja

Współczesne organizacje gromadzą ogromne ilości danych rejestrujących przebieg realizowanych procesów biznesowych. Dane te, zapisywane w postaci dzienników zdarzeń (ang. *event logs*), stanowią cenny zasób informacyjny, który umożliwia analizę i doskonalenie procesów z użyciem narzędzi eksploracji procesów (ang. *process mining*) [30]. Eksploracja procesów pozwala na odkrywanie modeli procesów wprost z danych historycznych, weryfikację zgodności rzeczywistego przebiegu procesu z jego modelowym opisem oraz wspomaganie decyzji w zakresie optymalizacji działalności organizacji.

Wraz z rosnącą popularnością systemów klasy ERP, CRM i BPM, wolumeny rejestrowanych zdarzeń osiągają rozmiary trudne do obsługi przez tradycyjne systemy zarządzania bazami danych. Duże organizacje, takie jak szpitale, instytucje finansowe czy przedsiębiorstwa produkcyjne, rejestrują na co dzień ogromne wolumeny zdarzeń, co prowadzi do powstawania gigantycznych zbiorów danych. Skuteczne przetwarzanie tak dużych zbiorów danych wymaga zastosowania skalowalnych technologii składowania oraz specjalizowanych języków zapytań dostosowanych do hierarchicznej struktury dzienników zdarzeń.

Standardem dla reprezentacji dzienników zdarzeń jest format XES (ang. *eXtensible Event Stream*), zdefiniowany w normie IEEE 1849-2016 [3]. Format ten opisuje trójpoziomową hierarchię danych: dziennik zawiera ślady (ang. *traces*), a każdy ślad zawiera zdarzenia (ang. *events*). Efektywne odpytywanie danych w takiej strukturze przy użyciu klasycznego SQL jest nietrywialnym zadaniem, wymagającym złożonych złączeń i podzapytań, co negatywnie wpływa zarówno na czytelność zapytań, jak i na wydajność ich wykonania.

Odpowiedzią na te wyzwania jest Process Query Language (PQL) [25], język zapytań opracowany w ramach projektu ProcessM [26] na Politechnice Poznańskiej. PQL został zaprojektowany z myślą o natywnej obsłudze hierarchicznej struktury dzienników zdarzeń i umożliwia intuicyjne formułowanie zapytań obejmujących jednocześnie poziomy logu, śladu i zdarzenia.

Dotychczasowa implementacja referencyjna PQL opiera się na relacyjnej bazie danych PostgreSQL[28]. Relacyjne bazy danych charakteryzują się jednak sztywnym, tabelarycznym modelem danych, który wymaga odgórnego zdefiniowania schematu i w naturalny sposób nie odzwierciedla hierarchicznej struktury dzienników XES.

Dokumentowe bazy danych NoSQL, takie jak Apache CouchDB [9], oferują alternatywny model składowania oparty na dokumentach JSON [11]. Podejście to jest znacznie bardziej elastyczne: dane nie muszą pasować do z góry narzuconego schematu, a ich struktura może swobodnie odzwierciedlać rzeczywisty kształt przechowywanych informacji. Hierarchia logu, śladu i zdarzenia

może być więc reprezentowana w sposób naturalny, bez konieczności rozbijania jej na wiele tabel i późniejszego sklejania wyników złączeniami. Bazy NoSQL charakteryzują się ponadto wysoką skalowalnością poziomą oraz wbudowanymi mechanizmami replikacji, co czyni je obiecującą platformą dla systemów eksploracji procesów przetwarzających duże wolumeny danych.

Niniejsza praca inżynierska podejmuje problem implementacji interpretera języka PQL dla dokumentowej bazy danych CouchDB. Temat ten jest istotny zarówno z perspektywy praktycznej, jako propozycja skalowalnej i elastycznej alternatywy dla implementacji relacyjnych, jak i naukowej, dostarczając danych na temat wydajności systemu w zależności od rozmiaru przetwarzanych danych.

## 1.2 Cel i zakres pracy

Nadrzędnym celem niniejszej pracy jest opracowanie i implementacja systemu zarządzania dziennikami zdarzeń obsługującego język Process Query Language (PQL) jako interfejs dostępu do danych, z wykorzystaniem dokumentowej bazy danych Apache CouchDB jako warstwy składowania. Opracowany system stanowi nakładkę na bazę danych, tłumaczącą zapytania zgodne ze specyfikacją PQL na język zapytań Mango Query [9] właściwy dla CouchDB.

Należy podkreślić, że PQL nie jest oryginalnym wkładem niniejszej pracy. PQL jest istniejącym, w pełni zdefiniowanym językiem zapytań opracowanym w ramach projektu ProcessM na Politechnice Poznańskiej. Niniejsza praca traktuje specyfikację PQL jako dane wejściowe i koncentruje się wyłącznie na problemie implementacji jego interpretera w oparciu o dokumentową bazę danych.

Główna hipoteza badawcza niniejszej pracy brzmi: *Możliwe jest zaimplementowanie w pełni funkcjonalnego interpretera języka PQL dla dokumentowej bazy danych CouchDB, który zapewni poprawność semantyczną wyników, zgodność ze specyfikacją języka oraz akceptowalną wydajność dla danych o rozmiarach typowych dla rzeczywistych zastosowań eksploracji procesów.*

Hipoteza ta będzie weryfikowana trójaspektowo: po pierwsze poprzez testy zgodności wyników zwracanych przez interpreter z wynikami oczekiwanymi zgodnie ze specyfikacją PQL, po drugie poprzez weryfikację niezmienności danych (sprawdzenie, czy każdy dziennik załadowany do bazy może zostać odtworzony w identycznej formie), a po trzecie poprzez eksperymenty wydajnościowe mierzące czas wykonania zapytań w zależności od rozmiaru przetwarzanych danych.

**W ramach niniejszej pracy zrealizowano następujące zadania:**

- implementacja parsera języka PQL z wykorzystaniem generatora parserów ANTLR4 [23], obejmującego pełną gramatykę języka PQL w wersji 0.4,
- opracowanie translatora zapytań PQL na język Mango Query, realizującego mapowanie klauzul `select, where, group by, order by, limit, offset` oraz `delete`,
- implementacja silnika wykonawczego zapytań, realizującego grupowanie, agregację, sortowanie, wyrażenia arytmetyczne oraz rozwiązywanie odwołań międzyzakresowych w pamięci aplikacji,
- implementacja strumieniowego importera dzienników XES do bazy CouchDB z obsługą formatów nieskompresowanych oraz skompresowanych (GZIP, ZIP), z zachowaniem oryginalnych identyfikatorów obiektów,
- implementacja eksportera danych z bazy CouchDB do formatu XES, umożliwiającego weryfikację niezmienności przetwarzanych danych,

- udostępnienie interfejsu REST umożliwiającego integrację systemu z zewnętrznym oprogramowaniem,
- przeprowadzenie eksperymentów wydajnościowych i analizy skalowalności systemu względem rozmiaru przetwarzanych danych.

## 1.3 Podział pracy

Podział zadań implementacyjnych i badawczych przedstawia się następująco:

**Konrad Maciejewski** był odpowiedzialny za:

- implementację warstwy składowania (I/O), komunikacji z bazą CouchDB oraz strumieniowego importu i eksportu dzienników zdarzeń XES,
- budowę silnika wykonawczego (*In-Memory Executor*) emulującego operacje SQL (m.in. wielowymiarowe grupowanie, agregacje, sortowanie),
- strukturyzację klas modelu zapytania w aplikacji i formatowanie wyników do standardu JSON,
- zaprojektowanie i przeprowadzenie eksperymentów badawczych (poprawność semantyczna, integralność danych, testy wydajności i skalowalności).

**Tsimafei Kotski** był odpowiedzialny za:

- projekt i implementację parsera języka PQL (przy użyciu generatora ANTLR4) oraz budowę abstrakcyjnego drzewa składniowego (AST),
- stworzenie translatora zapytań, tłumaczącego klauzule PQL na natywną składnię bazy CouchDB (*Mango Query*),
- wdrożenie mechanizmów walidacji zapytań od użytkownika oraz obsługę błędów składniowych.

## 1.4 Struktura pracy

Struktura pracy jest następująca. W rozdziale 2 przedstawiono problem interpretacji języka PQL, w tym opisano standard XES oraz specyfikację samego języka PQL. Rozdział 3 jest poświęcony przeglądowi literatury, w którym omówiono alternatywne języki zapytań i systemy baz danych. Rozdział 4 zawiera szczegóły techniczne dotyczące implementacji interpretera, w tym parsera i translatorów zapytań do języka Mango Query. W rozdziale 5 opisano przeprowadzone eksperymenty weryfikujące poprawność, niezmienność danych i wydajność stworzonego rozwiązania. Rozdział 6 to dyskusja nad uzyskanymi wynikami i cechami opracowanego interpretera. Rozdział 7 stanowi podsumowanie pracy i nakreśla dalsze kierunki jej rozwoju.

# Rozdział 2

# Problem interpretacji języka PQL

## 2.1 Standard eXtensible Event Stream

Format XES (ang. *eXtensible Event Stream*) jest standardem opracowanym przez IEEE Task Force on Process Mining i opublikowanym jako norma IEEE 1849-2016 [3]. Jego głównym celem jest zapewnienie jednolitego, opartego na XML formatu wymiany danych zdarzeń pomiędzy systemami rejestrującymi dane a narzędziami analitycznymi. Prekursorem dla XES był format MXML [32]. Jednak praktyczne doświadczenia ujawniły jego istotne ograniczenia: wszystkie dodatkowe atrybuty były traktowane jako wartości tekstowe bez jasno określonej semantyki, a nazewnictwo pojęć było niespójne. XES rozwiązał te problemy przez wprowadzenie systemu rozszerzeń nadających semantykę poszczególnym atrybutom.

Podstawą struktury XES jest trójpoziomowa hierarchia. Na najwyższym poziomie znajduje się log, który reprezentuje informacje związane z określonym procesem. Log zawiera listę śladów (ang. *traces*), gdzie każdy ślad odpowiada jednej instancji wykonania procesu. Wewnątrz śladu znajduje się uporządkowana lista zdarzeń (ang. *events*), z których każde reprezentuje pojedynczy, atomowy fakt zaobserwowany w ramach danej instancji. Kolejność zdarzeń w obrębie śladu jest istotna i odzwierciedla chronologiczny porządek obserwacji.

W praktyce dziennik odpowiada zazwyczaj jednemu procesowi biznesowemu (np. obsłudze zamówień), ślad reprezentuje jeden konkretny przypadek realizacji tego procesu (np. jedno zamówienie), a zdarzenia opisują kolejne kroki wykonane w ramach tego przypadku. Schemat tej hierarchii wygląda następująco:

```
log
 |- trace (przypadek 1)
 |   |- event (krok 1)
 |   \- event (krok 2)
 \- trace (przypadek 2)
     \- event (krok 1)
```

Informacje o każdym składniku przechowywane są w atrybutach. Standard definiuje siedem typów atrybutów, co przedstawiono w tabeli 2.1. Każdy klucz atrybutu musi być unikalny w obrębie tego samego składnika. Standard przewiduje opcjonalnie atrybuty zagnieżdżone (ang. *nested attributes*), których obsługa nie jest wymagana do zachowania zgodności ze standardem.

Atrybuty globalne (ang. *global attributes*) pozwalają deklarować, które atrybuty muszą być obecne we wszystkich śladach lub zdarzeniach w ramach danego logu. Pełnią one rolę wartości do-

| Typ XML | Typ danych | Opis |
|---|---|---|
| `string` | `xs:string` | Wartość tekstowa UTF-8 |
| `date` | `xs:dateTime` | Znacznik czasu UTC |
| `int` | `xs:long` | Liczba całkowita 64-bitowa |
| `float` | `xs:double` | Liczba zmiennoprzecinkowa 64-bitowa |
| `boolean` | `xs:boolean` | Wartość logiczna (`true`/`false`) |
| `id` | UUID (`string`) | Unikalny identyfikator UUID |
| `list` | lista atrybutów | Uporządkowana kolekcja atrybutów (typ złożony) |

Tabela 2.1: Typy atrybutów w standardzie XES

myślnych i ułatwiają narzędziom analitycznym przewidywanie struktury danych bez konieczności analizowania każdego zdarzenia z osobna.

Klasyfikatory (ang. *classifiers*) przypisują tożsamość każdemu zdarzeniu lub śladowi na podstawie wartości wybranego zestawu atrybutów. Przykładowo, klasyfikator oparty na atrybucie `concept:name` identyfikuje zdarzenia według nazwy aktywności, co stanowi podstawę większości algorytmów eksploracji procesów. Jeden log może definiować wiele klasyfikatorów, umożliwiając różne perspektywy analizy tych samych danych.

Kluczowym mechanizmem XES jest system rozszerzeń (ang. *extensions*). Rozszerzenie definiuje zestaw atrybutów dla określonych poziomów hierarchii i nadaje im interpretowalną semantykę. Każde rozszerzenie ma nazwę, prefiks i URI wskazujący na jego definicję; klucze atrybutów są poprzedzone prefiksem i dwukropkiem. Norma IEEE 1849-2016 definiuje siedem standardowych rozszerzeń, podsumowanych w Tabeli 2.2.

| Rozszerzenie | Prefiks | Główne atrybuty i ich przeznaczenie |
|---|---|---|
| Concept | `concept` | `concept:name` – ogólna nazwa składnika (nazwa aktywności, identyfikator przypadku) |
| Lifecycle | `lifecycle` | `lifecycle:transition` (model standardowy), `lifecycle:state` (model BPAF) – faza cyklu życia aktywności |
| Time | `time` | `time:timestamp` – znacznik czasu UTC zdarzenia; podstawa analiz temporalnych |
| Organizational | `org` | `org:resource`, `org:role`, `org:group` – zasób, rola i grupa organizacyjna odpowiedzialne za zdarzenie |
| Identity | `identity` | `identity:id` – unikalny identyfikator UUID dla logu, śladu, zdarzenia lub atrybutu |
| Cost | `cost` | `cost:total`, `cost:currency`, `cost:drivers` – informacje o kosztach powiązanych ze śladem lub zdarzeniem |
| Semantic | `semantic` | `semantic:modelReference` – odwołanie do zewnętrznego modelu ontologicznego, umożliwiające powiązanie atrybutów dziennika z pojęciami z dziedziny procesu |

Tabela 2.2: Standardowe rozszerzenia formatu XES

Serializacja XES opiera się na formacie XML ze schematem XSD dołączonym normatywnie do normy. Standard rozróżnia dwa poziomy zgodności: instancję ściśle zgodną (ang. *strictly conforming*), zawierającą wyłącznie elementy zdefiniowane w normie IEEE 1849-2016, oraz instancję zgodną (ang. *conforming*), która może zawierać dodatkowe elementy i rozszerzenia spoza standardu. Instancja ściśle zgodna gwarantuje pełną interoperacyjność między narzędziami, natomiast instancja zgodna pozwala na rozszerzanie formatu o własne atrybuty i rozszerzenia domenowo specyficzne, co jest powszechną praktyką w systemach produkcyjnych. W praktyce większość narzędzi eksploracji procesów obsługuje obie formy.

## 2.2 Język PQL

Process Query Language (PQL) to specjalizowany język zapytań [25]. PQL jest inspirowany standardem SQL, lecz zamiast operować na płaskich tabelach natywnie odwzorowuje trójpoziomową hierarchię XES i umożliwia jednoczesne odwoływanie się do atrybutów na różnych poziomach.

Wszystkie słowa kluczowe, identyfikatory i porównania wartości są wrażliwe na wielkość liter. PQL definiuje trzy zakresy (ang. *scopes*): `log` (skrót `l`), `trace` (skrót `t`) oraz `event` (skrót `e`). Jeśli zakres nie jest podany jawnie, domyślnie stosowany jest zakres `event`. Język rozróżnia pięć typów danych, zestawionych w tabeli 2.3. Każdy typ danych obejmuje specjalną wartość `null` reprezentującą brak wartości. PQL nie obsługuje jawnych rzutowań typów.

| **Typ** | **Format / standard** | **Opis** |
|---|---|---|
| `uuid` | ISO/IEC 9834-8:2014 [2] | Unikalny identyfikator w formacie 8-4-4-4-12 (hex) |
| `string` | UTF-8 [35] | Tekst; obsługuje sekwencje ucieczki (np. `\n`, `\t`) |
| `number` | IEEE 754-2019 [4] | Liczba zmiennoprzecinkowa; zapis dziesiętny i naukowy |
| `datetime` | ISO 8601-1:2019 [5] | Znacznik czasu UTC z dokładnością do milisekund |
| `boolean` | `true` / `false` | Wartość logiczna; porównania z `null` dają `false` |

Tabela 2.3: Typy danych w języku PQL

### Gramatyka

PQL posiada formalną gramatykę bezkontekstową zapisaną w formacie rozszerzonego BNF (EBNF) z rozszerzeniami ANTLR4. Gramatyka dzieli się na warstwę leksykalną definiującą tokeny oraz warstwę parsera budującą drzewo AST. Podstawowa reguła parsera prezentuje się następująco:

```
query        : read_query EOF | delete_query EOF
read_query   : select where? group_by? order_by? limit? offset?
delete_query : delete where? order_by? limit? offset?
```

Pytajnik (`?`) oznacza opcjonalność. Kolejność klauzul jest ściśle określona. Puste zapytanie jest w pełni poprawne i równoznaczne z `select *`, zwracając wszystkie dane bez przetwarzania.

### Wynoszenie zakresów

Mechanizm wynoszenia zakresów (ang. *scope hoisting*), oznaczany prefiksem `^`, przesuwa wyrażenie do zakresu nadrzędnego: `^e:timestamp` przenosi wyrażenie ze zdarzenia do śladu, a `^^e:name` aż do logu. Hoistingu nie można stosować ponad poziom logu. Ponieważ relacja rodzic-dziecko jest jeden-do-wielu, przeniesione wyrażenie reprezentuje listę wartości. W `where` umożliwia filtrowanie śladów według wartości atrybutów zdarzeń, w `group by` pozwala grupować ślady według sekwencji aktywności (warianty procesu). W `select` i `order by` hoisting jest dopuszczalny wyłącznie jako argument funkcji agregującej.

### Semantyka klauzul

Klauzula `select` określa zwracane atrybuty. Występuje w trzech formach: `select *` zwraca wszystkie atrybuty, `select scope:*` zwraca atrybuty wybranego zakresu `scope`, a trzecia forma pozwala wybierać konkretne wyrażenia arytmetyczne i funkcje agregujące. Klauzula `delete` usuwa

wskazane składniki kaskadowo: usunięcie logu pociąga za sobą usunięcie wszystkich jego śladów i zdarzeń. `delete` nie może wystąpić razem z `select` ani `group by`.

Klauzula `where` filtruje składniki według wyrażenia logicznego. Zastosowanie wynoszenia zmienia semantykę: `where ^e:name = 'X'` wybiera ślady posiadające przynajmniej jedno zdarzenie o danej nazwie, zwracając jednak wszystkie zdarzenia tego śladu.

Klauzula `group by` grupuje składniki według wartości podanych atrybutów. Po jej zastosowaniu w `select` i `order by` dostępne są tylko atrybuty wymienione w `group by`; pozostałe mogą być użyte jako argumenty funkcji agregującej. Jeśli w `select` użyto funkcji agregującej bez jawnego `group by`, stosowane jest niejawne grupowanie (ang. *implicit group by*).

Klauzula `order by` sortuje wyniki według listy wyrażeń z kierunkiem `asc` lub `desc`. Wartość `null` traktowana jest jako większa od wszystkich innych. `limit` ogranicza liczbę zwracanych składników, a `offset` pomija początkowe rekordy.

### Operatory i obsługa wartości null

PQL udostępnia operatory arytmetyczne (`+`, `-`, `*`, `/`), porównania (`=`, `!=`, `<`, `<=`, `>`, `>=`), logiczne (`and`, `or`, `not`), tekstowe `like` oraz `matches`. Kluczową regułą jest to, że każde porównanie z `null`, niezależnie od operatora, zwraca `false` (z wyjątkiem `is null` i `is not null`).

### Funkcje i ograniczenia

PQL udostępnia pięć funkcji agregujących (`min`, `max`, `avg`, `count`, `sum`) oraz funkcje skalarne. Do znanych ograniczeń należy brak bezpośrednich odwołań do zagnieżdżonych atrybutów oraz brak obsługi elementów atrybutu listy poza klauzulą `select`.

## 2.3 Problem interpretacji języka PQL

Przedmiotem niniejszej pracy jest problem interpretacji języka PQL w środowisku dokumentowej bazy danych NoSQL. Formalnie, wejściem systemu jest łańcuch znaków stanowiący zapytanie w języku PQL oraz baza danych Apache CouchDB zawierająca dzienniki zdarzeń zaimportowane z plików XES. Wyjściem systemu jest hierarchicznie ustrukturyzowana kolekcja logów, śladów i zdarzeń spełniająca semantykę zapytania PQL, zwracana przez interfejs REST lub wyeksportowana do formatu XES.

Część operacji PQL można względnie łatwo odwzorować na możliwości Mango Query (natywnego języka zapytań bazy CouchDB). Przykładowo, proste filtrowanie na poziomie zdarzenia mapuje się bezpośrednio na selektor Mango, projekcja wybranych atrybutów odpowiada parametrowi `fields`, a stronicowanie odwzorowuje się na parametry `limit` i `skip`. Poważniejsze trudności wynikają z braku w Mango Query odpowiednika relacyjnych złączeń (`join`) oraz klauzuli `group by`. Problemy te można podzielić na cztery kategorie:

- **Zapytania wielozakresowe (cross-scope):** Mango nie potrafi odwoływać się do atrybutów powiązanych dokumentów. Zapytanie z wynoszeniem wymaga wieloetapowego przetwarzania: pobrania identyfikatorów śladów, a następnie filtrowania dokumentów na ich podstawie.

- **Grupowanie i agregacja:** Wszystkie operacje grupowania muszą być realizowane w pamięci aplikacji. Szczególnie kosztowne jest grupowanie według wariantów procesu, wymagające porównania pełnych sekwencji zdarzeń.

- **Mapowanie nazw atrybutów:** PQL posługuje się skróconymi nazwami atrybutów (np. `name` zamiast `concept:name`), natomiast dokumenty w CouchDB przechowują oryginalne klucze XES. Interpreter musi utrzymywać tablicę mapowań i uwzględniać fakt, że ten sam atrybut odwzorowuje się na różne ścieżki JSON w zależności od zakresu.
- **Semantyka wartości null:** W CouchDB brak pola i pole o wartości `null` są interpretowane różnie, podczas gdy PQL traktuje oba przypadki jednakowo. Wymaga to dodatkowej obsługi logicznej po stronie interpretera.

**Pomiar jakości rozwiązania obejmuje trzy aspekty:**

1. **Poprawność semantyczna:** Wyniki zwracane przez interpreter muszą być w pełni zgodne ze specyfikacją PQL, co jest weryfikowane testami jednostkowymi i integracyjnymi.
2. **Niezmienność danych:** Każdy załadowany do bazy dziennik można wyeksportować do identycznego pliku XES z zachowaniem oryginalnych identyfikatorów.
3. **Wydajność obliczeniowa:** Mierzona czasem wykonania w zależności od rozmiaru danych (z uwzględnieniem wyższej złożoności zapytań z hoistingiem i grupowaniem).

# Rozdział 3

# Przegląd literatury

Projektowanie systemu zarządzania dziennikami zdarzeń z obsługą języka PQL wymagało przeanalizowania istniejących rozwiązań w dwóch głównych obszarach: języków zapytań dedykowanych dla dzienników zdarzeń oraz platform bazodanowych, na których można przechować dane w formacie XES. Poniższy przegląd uwzględnia zarówno opis alternatywnych podejść, jak i ich porównanie z rozwiązaniem zaproponowanym w niniejszej pracy.

## 3.1 Języki zapytań do dzienników zdarzeń

Analiza istniejących języków zapytań dedykowanych dla dzienników zdarzeń została przeprowadzona na podstawie pracy [25], która systematycznie porównuje dostępne DSL (ang. *domain-specific language*) według 14 cech jakościowych, takich jak posiadanie formalnej gramatyki, deklaratywność, wysoki poziom abstrakcji, kompletność domenowa, przenaszalność, otwartość dokumentacji czy wsparcie jakości kodu.

Poniżej przedstawiono charakterystykę najważniejszych alternatyw i ich porównanie z językiem PQL:

**DAPOQL** [12, 13] Jest językiem deklaratywnym opartym na algebrze zbiorów, obsługującym predykatowe filtrowanie, funkcje temporalne i schematy wielologowe. Spełnia większość cech jakościowych określonych w [25], jednak jego kompletna dokumentacja przestała być publicznie dostępna. W porównaniu do PQL, DAPOQL nie posiada otwartej i aktualnej specyfikacji, a jego projekt wydaje się zawieszony.

**FPSPARQL** [10] Rozszerza grafowy język zapytań SPARQL na potrzeby przetwarzania dzienników zdarzeń. Dokumentacja FPSPARQL nie posiada formalizmu wymaganego do jednoznacznej implementacji: brakuje gramatyki i matematycznego sformułowania semantyki. W odniesieniu do naszego rozwiązania PQL oferuje wyższy poziom formalizacji i większą dostępność dokumentacji.

**PIQL** [27] Jest rozszerzeniem standardu DMN (*Decision Model and Notation*) umożliwiającym wyodrębnianie kluczowych wskaźników wydajności (KPI). Choć kod PIQL przypomina język naturalny, ma on wąski zakres zastosowania ograniczony do obliczania KPI. PQL oferuje znacznie szerszy zakres funkcjonalny obejmujący filtrowanie, agregację, grupowanie i usuwanie danych.

**QUELI** [7] Język zorientowany na wykrywanie wad jakościowych w dziennikach zdarzeń. Cel QUELI jest wyspecjalizowany w kwestii jakości danych, co różni go zasadniczo od PQL, który skupia się na ogólnym przetwarzaniu i odpytywaniu dzienników.

**XTemp** [14] DSL oparty na XSLT i XPath, przeznaczony do przetwarzania dzienników w formacie XML (w tym XES). Jest funkcjonalnie bogaty, ale jego główną wadą jest wymaganie dziesiątek linii kodu do wykonania prostych zadań. PQL umożliwia wyrażenie tych samych operacji za pomocą zaledwie kilku klauzul.

**Celonis PQL** [34] Dojrzały komercyjny język zapytań z ponad 150 operatorami ukierunkowanymi na analizę biznesową. Mimo dojrzałości, gramatyka tego języka nie jest publicznie dostępna, co utrudnia niezależną implementację. Nasz system oparty na otwartym języku PQL stosuje model zgodny ze standardem XES oraz posiada pełną, publiczną specyfikację.

**IQL** [29] Język oparty na SQL, rozszerzony o operatory sekwencyjne. Jego referencyjna implementacja jest jednak niekompletna (brakuje m.in. parsera języka), co praktycznie uniemożliwia jego użycie. W przeciwieństwie do IQL, PQL oferuje kompletną implementację zrealizowaną w projekcie ProcessM, z pełnym parserem wygenerowanym przez ANTLR4.

Poniższa tabela prezentuje zestawienie omówionych języków według 14 kryteriów jakościowych.

| DSL | Grammar-based | Declarative | High-level | Focused | Complete | Portable | Self-documenting | Well-documented | Open | Familiar | Supporting code quality | Maintainable | Practical use | Scalable |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DSLs for querying event logs | | | | | | | | | | | | | | |
| DAPOQL | ?[1] | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | ?[1] | | ✓ | | ✓ | ✓ |
| FPSPARQL | | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | | | | ✓ | ✓ | ✓ |
| PIQL | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | ✓ | | ?[2] | ✓ | ✓ | |
| QUELI | | ?[3] | ?[3] | ✓ | | ?[3] | ?[3] | | | ?[3] | ?[3] | ?[3] | | ?[3] |
| XTemp | ✓ | | ✓ | | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Celonis PQL | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | ✓ | ✓ | ✓ | ✓ | ✓ |
| IQL | | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | ✓ | ✓ | ✓ | | ✓ | ✓ |
| **PQL (this work)** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

Tabela 3.1: Porównanie cech istniejących języków DSL [25]

?[1] Adres URL repozytorium języka nie jest dostępny.
?[2] Wsparcie funkcjonalności zależy od narzędzia.
?[3] Brak wystarczających szczegółów.

Zestawienie cech omawianych języków pokazuje, że PQL jako jedyny z nich spełnia wszystkie zdefiniowane kryteria badawcze. Niniejsza praca wpisuje się w ten kontekst jako pierwsza implementacja PQL dla środowiska NoSQL, poszerzając dostępność języka poza ekosystem relacyjny.

## 3.2 Systemy baz danych dla dzienników zdarzeń

Wyboru systemu bazodanowego dla dzienników XES dokonuje się w kontekście kilku powiązanych wymagań: elastycznego modelu danych dopasowanego do hierarchii log–ślad–zdarzenie, wystarczającej wyrazistości języka zapytań, wydajności przy dużych wolumenach danych oraz skalowalności horyzontalnej. Poniżej omówiono główne klasy systemów i porównano je z wybranym w tej pracy Apache CouchDB [9].

### Relacyjne bazy danych i schemat RXES

Referencyjna implementacja ProcessM opiera się na bazie PostgreSQL. Dla relacyjnego przechowywania dzienników XES opracowano schemat RXES (*Relational XES*) [25], który jest wstecznie kompatybilny z XES. Implementacja ta jest jednak wyraźnie wolniejsza przy dostępie do danych zdarzeń. Dodatkowym ograniczeniem jest sztywny schemat tabelaryczny: hierarchia musi być rozbita na powiązane tabele, a odpytywanie wymaga kosztownych operacji `join`. Atrybuty o zmiennych kluczach wymagają modelu EAV (*Entity-Attribute-Value*) lub kolumn JSON, co komplikuje zapytania.

### Grafowe bazy danych (Neo4j)

Część prac badawczych [15, 19] zaleca przechowywanie dzienników w grafowych bazach danych, np. Neo4j [22] i odpytywanie ich specjalizowanych językiem, np. Cypher dla Neo4j. Model grafowy naturalnie odwzorowuje relacje następstwa zdarzeń. Cypher został zaprojektowany do odpytywania grafów, a nie dzienników zdarzeń, co czyni go trudniejszym w użyciu dla analityków. Dla porównania, PQL jest łatwiejszy do przyswojenia przez użytkowników znających tradycyjny SQL.

### Dokumentowe bazy danych NoSQL (MongoDB)

MongoDB [21] to najbliższa dokumentowa alternatywa dla CouchDB. Oba systemy opierają się na elastycznych schematach (JSON/BSON) i skalowaniu horyzontalnym. Choć MongoDB dysponuje natywnym *Aggregation Framework*, który wydajnie realizuje operacje `group by` i `order by`, posiada w naszym kontekście dwa kluczowe ograniczenia. Po pierwsze, w przeciwieństwie do CouchDB, nie eksponuje natywnego interfejsu REST API [16], co wymusza budowę dodatkowej warstwy pośredniczącej. Po drugie, korzysta z licencji SSPL [20], nieuznawanej przez organizację OSI za otwarte oprogramowanie (*Open Source*). Ten konflikt z wymogiem pełnej otwartości projektu ostatecznie wykluczył MongoDB z zastosowania.

### Kolumnowe bazy danych (Apache Cassandra)

Apache Cassandra [8] to kolumnowa baza danych NoSQL przeznaczona dla systemów o masowej skali. Z perspektywy XES posiada ona jednak poważne ograniczenia: model kolumnowy nie odwzorowuje naturalnie hierarchii procesu, język CQL (*Cassandra Query Language*) nie obsługuje złożonych złączeń ani klauzuli `group by` dla dowolnych wyrażeń, a schemat musi być z góry zdefiniowany pod kątem zapytań (*query-first design*).

### Apache CouchDB jako optymalny wybór

Apache CouchDB został wybrany jako środowisko docelowe dla interpretera PQL ze względu na ścisłe dopasowanie jego modelu danych do wymagań systemu. Dokumenty JSON mogą naturalnie odwzorowywać trójpoziomową hierarchię XES. Brak sztywnego schematu oznacza, że różne zdarzenia w tym samym logu mogą posiadać różne zestawy atrybutów, co jest właściwością standardu XES. CouchDB jest systemem *open-source* z wbudowanym REST API, co upraszcza architekturę warstwy dostępu i eliminuje potrzebę dodatkowych sterowników, a jego język zapytań Mango Query oferuje dostateczną wyrazistość dla potrzeb filtrowania i projekcji.

Tabela 3.2 prezentuje zestawienie porównawcze omawianych systemów pod kątem wymagań interpretera PQL.

| **System** | **Model danych** | **Język zapytań** | **Schemat** | **Skalowanie** | **Ocena dla PQL** |
|---|---|---|---|---|---|
| PostgreSQL / RXES | Relacyjny | SQL | Sztywny | Wertykalne | Wysoka złożoność `join`, trudna obsługa zmiennych atrybutów |
| Neo4j | Grafowy | Cypher | Elastyczny | Horyzontalne | Niededykowany dla XES; wysoka bariera wejścia |
| MongoDB | Dokumentowy | MQL | Elastyczny | Horyzontalne | Wydajne agregacje, brak natywnego REST API, bariera licencyjna (SSPL) |
| Apache Cassandra | Kolumnowy | CQL | Sztywny | Horyzontalne | Brak `group by`, złączeń; projektowanie typu *query-first* |
| Apache CouchDB | Dokumentowy | Mango Query | Elastyczny | Horyzontalne | Najlepsze dopasowanie: natywne REST API, model JSON |

Tabela 3.2: Zestawienie porównawcze systemów bazodanowych pod kątem interpretera PQL

Analiza powyższych systemów potwierdza, że Apache CouchDB jest najbardziej odpowiednim środowiskiem dla implementacji interpretera PQL spośród analizowanych alternatyw w ramach niniejszej pracy. Choć MongoDB oferuje wyższą wydajność agregacji, to CouchDB przewyższa go w zakresie natywnej dostępności REST API oraz, co najważniejsze, posiada w pełni otwartą licencję Apache 2.0 [1], zgodną z założeniami projektu. Relacyjne bazy danych (PostgreSQL) są trudniejsze w obsłudze dla zmiennych atrybutów XES i wymagają kosztownych złączeń `join`. Grafowe bazy danych, chociaż naturalnie modelują relacje następstwa, wymagają języków zapytań nieznajomych użytkownikowi SQL. Cassandra jest najsilniejsza przy dużych wolumenach zapisu, ale jej ograniczone możliwości zapytań *ad-hoc* czynią ją nieodpowiednią dla pełnej implementacji semantyki PQL.

# Rozdział 4

# Interpreter PQL

Niniejszy rozdział stanowi szczegółowy opis implementacji interpretera języka PQL dla dokumentowej bazy danych Apache CouchDB. Interpreter jest centralnym komponentem opracowanego systemu i realizuje pełny cykl przetwarzania zapytania: od analizy składniowej tekstu PQL, przez translację na język zapytań Mango Query, aż po wykonanie zapytania na bazie danych i sformatowanie wyników w postaci zgodnej ze standardem XES. System został zaimplementowany w języku Kotlin [6] z wykorzystaniem narzędzia budowania Gradle [18] i uruchamiany jest jako aplikacja Spring Boot [33] eksponująca interfejs REST API [16].

Architektura interpretera jest zorganizowana w cztery główne warstwy przetwarzania, które odpowiadają strukturze niniejszego rozdziału. Warstwa parsera (Sekcja 4.1) odpowiada za analizę leksykalną i składniową tekstu zapytania PQL oraz budowę abstrakcyjnego drzewa składniowego (AST). Warstwa translatora (Sekcja 4.2) przekształca AST w zapytanie Mango Query zrozumiałe dla CouchDB, realizując jednocześnie mapowanie nazw atrybutów PQL na ścieżki pól w dokumentach JSON. Warstwa wykonawcza (Sekcja 4.3) koordynuje całość procesu wykonania zapytania, obejmując rozwiązywanie warunków międzyzakresowych, ewaluację warunków w pamięci, sortowanie, grupowanie, agregację oraz formatowanie wyników. Warstwa wydajnościowa (Sekcja 4.4) opisuje mechanizmy optymalizacji zastosowane w celu zapewnienia akceptowalnej wydajności systemu.

Oprócz samego interpretera zapytań, system obejmuje również strumieniowy importer dzienników XES do bazy CouchDB, eksporter danych z bazy do formatu XES oraz interfejs REST API. Komponenty te są opisane w odpowiednich podsekcjach Sekcji 4.3, ponieważ stanowią integralną część procesu wykonywania zapytań i zarządzania danymi.

## 4.1 Parser PQL

Parser PQL jest odpowiedzialny za transformację tekstowej postaci zapytania w strukturę danych zrozumiałą dla dalszych etapów przetwarzania. Implementacja parsera opiera się na generatorze parserów ANTLR4 (*ANother Tool for Language Recognition*) [23], który na podstawie formalnej gramatyki bezkontekstowej automatycznie generuje kod analizatora leksykalnego (leksera) oraz analizatora składniowego (parsera).

### Gramatyka leksykalna

Lekser zdefiniowany w pliku `src/main/antlr/QLLexer.g4` rozpoznaje tokeny języka PQL. Gramatyka leksykalna definiuje następujące kategorie tokenów:

- słowa kluczowe klauzul: `select`, `delete`, `where`, `group by`, `order by`, `limit`, `offset`, przy czym `group by` i `order by` są traktowane jako pojedyncze tokeny pomimo obecności separatora (białych znaków);
- identyfikatory zakresów (`SCOPE`): `log` (lub `l`), `trace` (lub `t`), `event` (lub `e`), umożliwiające precyzyjne wskazanie poziomu hierarchii XES;
- literały: `STRING` (w apostrofach lub cudzysłowach), `NUMBER` (z opcjonalną częścią dziesiętną i wykładnikiem), `BOOLEAN` (`true`/`false`), `DATETIME` (z prefiksem `D` i formatem zgodnym z ISO 8601), `UUID` oraz `NULL`;
- funkcje agregujące (`FUNC_AGGR`): `min`, `max`, `avg`, `count`, `sum`, rozpoznawane przez lekser wraz z otwierającym nawiasem, co eliminuje dwuznaczność z identyfikatorami o tej samej nazwie;
- funkcje skalarne jednoargumentowe (`FUNC_SCALAR1`): `date`, `time`, `year`, `month`, `day`, `hour`, `minute`, `second`, `millisecond`, `quarter`, `dayofweek`, `upper`, `lower`, `round`;
- funkcja skalarna bezargumentowa (`FUNC_SCALAR0`): `now`;
- operatory: arytmetyczne (`*`, `/`, `+`, `-`), porównania (`<`, `<=`, `=`, `!=`, `>`, `>=`), logiczne (`and`, `or`, `not`), przynależności (`in`, `not in`), dopasowania (`matches`, `like`) oraz testowania wartości `null` (`is null`, `is not null`).

Istotnym elementem gramatyki leksykalnej jest fragment `SCOPE_PREFIX`, który definiuje opcjonalny prefiks zakresu i znaczniki hoistingu. Każdy identyfikator, literał i funkcja mogą być poprzedzone prefiksem zakresu (np. `e:`, `t:`, `l:`) oraz znakami hoistingu (`^` lub `^^`). Mechanizm ten jest realizowany na poziomie leksera, co pozwala na jednoznaczne rozpoznanie zakresu atrybutu już podczas analizy leksykalnej.

### Gramatyka składniowa

Parser zdefiniowany w pliku `src/main/antlr/QLParser.g4` opisuje strukturę składniową zapytań PQL. Aksjomatem gramatyki jest reguła `query`, która rozróżnia dwa typy zapytań: `read_query` (zapytanie odczytujące z klauzulą `select`) oraz `delete_query` (zapytanie usuwające z klauzulą `delete`). Struktura zapytania odczytującego obejmuje opcjonalne klauzule `select`, `where`, `group by`, `order by`, `limit` i `offset`, ułożone w ściśle określonej kolejności.

Wyrażenia arytmetyczne (`arith_expr`) są definiowane rekurencyjnie z uwzględnieniem priorytetów operatorów: mnożenie i dzielenie mają wyższy priorytet niż dodawanie i odejmowanie. Wyrażenia logiczne (`logic_expr`) również respektują priorytety: `not` ma najwyższy priorytet, następnie `and`, a najniższy `or`. Kolejność produkcji w gramatyce odzwierciedla te priorytety, co gwarantuje poprawną budowę drzewa składniowego.

### Wrażliwość na wielkość liter (Case-sensitivity)

Zgodnie z formalną specyfikacją języka PQL, wszystkie słowa kluczowe, identyfikatory oraz porównania wartości muszą być bezwzględnie wrażliwe na wielkość liter (*case-sensitive*). Wymóg ten jest w pełni egzekwowany przez opracowany system, bazując na naturalnej właściwości parsera ANTLR4, który domyślnie rozróżnia wielkość znaków podczas analizy leksykalnej. Architektura interpretera celowo nie stosuje żadnych strumieniowych mechanizmów normalizacji tekstu (takich jak wymuszanie *lowercase*), co gwarantuje pełną zgodność z referencyjnym standardem. Słowa kluczowe są ściśle weryfikowane przez lekser zgodnie z ich definicją (np. `select`, `where`), a klasa

`PqlFieldMapper` precyzyjnie zachowuje oryginalną pisownię niestandardowych atrybutów biznesowych, przekazując je w nienaruszonej formie do zapytań Mango, co jest krytyczne dla poprawnego filtrowania danych w bazie CouchDB.

### Budowa modelu zapytania (AST)

Klasa `PqlQueryListener` implementuje wzorzec *listener* z biblioteki ANTLR4 i jest odpowiedzialna za budowę abstrakcyjnego drzewa składniowego w trakcie przechodzenia drzewa rozbioru. *Listener* przetwarza poszczególne węzły drzewa składniowego i tworzy odpowiadające im obiekty modelu danych:

- `PqlQuery` dla zapytań odczytujących, zawierający informacje o kolekcji docelowej (zakresie), projekcjach, warunkach, sortowaniu, limitach, offsetach i grupowaniu;
- `PqlDeleteQuery` dla zapytań usuwających, zawierający zakres usuwania, warunki, sortowanie, limit i offset;
- `PqlProjection` reprezentujący pojedynczą kolumnę w klauzuli `select`, z informacjami o zakresie, atrybucie, funkcji lub wyrażeniu arytmetycznym;
- `PqlCondition` jako zapieczętowana hierarchia klas (*sealed class*) modelująca warunki logiczne: `Simple` (warunek prosty), `And`, `Or`, `Not` oraz `AlwaysFalse` (dla warunków statycznie fałszywych, np. `0 = 1`);
- `PqlArithmeticExpression` jako zapieczętowana hierarchia klas reprezentująca wyrażenia arytmetyczne: `Binary` (operacja dwuargumentowa), `Attribute` (odwołanie do atrybutu), `Literal` (wartość stała) i `FunctionCall` (wywołanie funkcji).

*Listener* automatycznie wykrywa docelowy zakres zapytania na podstawie najgłębszego użytego prefiksu atrybutu (w przypadku braku – domyślnie `event`). Ustalenie tego zakresu determinuje cały plan wykonania zapytania. Po pierwsze, definiuje on bazową kolekcję poprzez wstrzyknięcie warunku `docType` do zapytania Mango. Po drugie, stanowi punkt odniesienia dla klauzuli `where`: warunki zgodne z zakresem są tłumaczone na natywne selektory CouchDB, natomiast odwołania do wyższych hierarchii stają się warunkami międzyzakresowymi (*cross-scope*), wymagającymi ewaluacji dwuprzebiegowej. Analogicznie, w przypadku `select` i `group by`, docelowy zakres wskazuje, które atrybuty odczytywane są z dokumentu bazowego, a które wymagają pobrania lub wstrzyknięcia danych z dokumentów nadrzędnych.

### Obsługa błędów składniowych

Klasa `ThrowingErrorListener` zastępuje domyślny mechanizm obsługi błędów ANTLR4 i rzuca wyjątek `IllegalArgumentException` z komunikatem wskazującym numer linii, pozycję znaku i opis błędu składniowego. Dzięki temu niepoprawne zapytania są odrzucane natychmiast na etapie parsowania, bez uruchamiania dalszych etapów przetwarzania (mechanizm *fail-fast*).

## 4.2 Translator PQL do Mango Query

Translator stanowi warstwę pośrednią między modelem zapytania PQL a natywnym językiem zapytań bazy CouchDB. Jego zadaniem jest przekształcenie obiektów `PqlQuery` i `PqlDeleteQuery` w dokumenty JSON zgodne ze składnią Mango Query, które mogą być bezpośrednio wykonane na endpoincie `_find` bazy CouchDB.

### Struktura dokumentów w CouchDB

Fundamentalną decyzją projektową systemu jest sposób odwzorowania trójpoziomowej hierarchii XES (log, trace, event) na dokumenty JSON w bazie CouchDB. Przyjęto model z trzema typami dokumentów, rozróżnianymi za pomocą pola `docType`:

| *Typ dokumentu* | *Pole w CouchDB* | *Opis* |
|---|---|---|
| `log` | `_id` | identyfikator w formacie `log_UUID` |
| | `docType` | stała wartość „log” |
| | `source` | techniczny identyfikator pochodzenia danych, przechowujący nazwę pliku źródłowego |
| | `importTimestamp` | znacznik czasu importu |
| | `log_attributes` | obiekt z globalnymi metadanymi całego procesu |
| `trace` | `_id` | identyfikator w formacie `logId_traceUUID` |
| | `docType` | stała wartość „trace” |
| | `logId` | referencja do dokumentu logu |
| | `xes_attributes` | obiekt z lokalnymi atrybutami biznesowymi pojedynczej instancji (śladu) |
| `event` | `_id` | identyfikator w formacie `logId_eventUUID` |
| | `docType` | stała wartość „event” |
| | `logId` | referencja do dokumentu logu |
| | `traceId` | referencja do dokumentu śladu |
| | `activity` | nazwa aktywności (odpowiada `concept:name`) |
| | `timestamp` | znacznik czasu zdarzenia (odpowiada `time:timestamp`) |
| | `eventIndex` | kolejność zdarzenia w śladzie |
| | `xes_attributes` | obiekt z lokalnymi atrybutami biznesowymi konkretnego kroku (zdarzenia) |
| | `_types` | techniczny obiekt JSON przechowujący oryginalne mapowanie typów XES dla atrybutów w celu ich bezstratnej rekonstrukcji |

Tabela 4.1: Struktura i zawartość dokumentów w bazie CouchDB

Dodatkowo zdarzenie zawiera zdenormalizowane pola z wyższych zakresów: `l:concept:name` (nazwa logu), `t:concept:name` (nazwa śladu) oraz wartości klasyfikatorów (pola z prefiksem `c:`). Wszystkie te wartości są wyliczane i wstrzykiwane w locie już na etapie strumieniowego importu (parsowania) pliku XES, zanim finalny dokument JSON zostanie zapisany w bazie danych. Denormalizacja danych z wyższych zakresów do poziomu zdarzenia jest kluczową optymalizacją, która eliminuje konieczność wykonywania dodatkowych zapytań (`join`-ów) dla najczęstszych operacji, takich jak filtrowanie zdarzeń po nazwie śladu lub logu. Szczegółowa analiza narzutu pamięciowego (*storage overhead*) wynikającego z tego kompromisu architektonicznego została przedstawiona w rozdziale poświęconym eksperymentom.

### Mapowanie nazw atrybutów (`PqlFieldMapper`)

Klasa `PqlFieldMapper` realizuje translację między nazewnictwem PQL a ścieżkami pól w dokumentach JSON. Specyfikacja PQL definiuje skrócone nazwy standardowych atrybutów XES (np. `name` zamiast `concept:name`, `timestamp` zamiast `time:timestamp`, `resource` zamiast `org:resource`). `PqlFieldMapper` utrzymuje słownik tych aliasów i rozwiązuje je do pełnych nazw. Ponadto klasa

ta buduje ścieżki dostępu (`FieldPath`) uwzględniające strukturę dokumentu na danym poziomie hierarchii. Na przykład atrybut `name` na poziomie zdarzenia jest mapowany na ścieżkę `activity` (pole bezpośrednie), podczas gdy ten sam atrybut na poziomie śladu jest mapowany na ścieżkę `xes_attributes.concept:name` (pole w obiekcie zagnieżdżonym). Decyzja o wyodrębnieniu nazwy zdarzenia na główny poziom dokumentu podyktowana była wymogami optymalizacyjnymi. Ponieważ aktywność stanowi fundamentalny wymiar analityczny w dziedzinie *Process Mining* (wykorzystywany do masowego filtrowania i grupowania wariantów), relokacja ta umożliwia założenie wysoce wydajnego indeksu B-drzewa w bazie CouchDB, podczas gdy nazwa śladu pozostaje wewnątrz zagnieżdżonego ładunku biznesowego. Tabela 4.2 przedstawia wybrane mapowania atrybutów dla różnych zakresów.

| **Atrybut PQL** | **Zakres** | **Ścieżka w dokumencie CouchDB** |
|---|---|---|
| `name` / `concept:name` | `event` | `activity` |
| `name` / `concept:name` | `trace` | `xes_attributes.concept:name` |
| `name` / `concept:name` | `log` | `log_attributes.concept:name` |
| `timestamp` / `time:timestamp` | `event` | `timestamp` |
| `id` / `identity:id` | `event` | `xes_attributes.identity:id` |
| `id` / `identity:id` | `trace` | `xes_attributes.identity:id` |
| `id` / `identity:id` | `log` | `log_attributes.identity:id` |
| `resource` / `org:resource` | `event` | `xes_attributes.org:resource` |

Tabela 4.2: Mapowanie wybranych atrybutów PQL na ścieżki w CouchDB

### Translacja warunków na selektory Mango

Klasa `PqlToCouchDbTranslator` przekształca warunki PQL (obiekty `PqlCondition`) na selektory JSON zgodne ze składnią Mango Query. Translacja odbywa się rekurencyjnie, zgodnie ze strukturą drzewa warunków:

- warunek prosty (`PqlCondition.Simple`) jest tłumaczony na obiekt JSON z kluczem odpowiadającym ścieżce pola i wartością będącą operatorem Mango (np. `$eq`, `$ne`, `$gt`, `$lt`, `$gte`, `$lte`, `$in`, `$nin`, `$regex`, `$exists`);
- koniunkcja (`PqlCondition.And`) jest tłumaczona na operator `$and` z tablicą podwarunków;
- alternatywa (`PqlCondition.Or`) jest tłumaczona na operator `$or`;
- negacja (`PqlCondition.Not`) jest tłumaczona kontekstowo: negacja równości staje się nierównością (`$ne`), negacja `in` staje się `not in` (`$nin`), a negacja bardziej złożonych warunków wykorzystuje operator `$nor`;
- wewnętrzny znacznik techniczny `AlwaysFalse` (wynikający z uwarunkowań strukturalnych, np. pustej listy `in`, a nie z analizy spełnialności wyrażeń logicznych) jest tłumaczony na selektor `_id = ,,__ALWAYS_FALSE__''`. Zapewnia to natychmiastowe odrzucenie zapytania przez główny indeks bazy w czasie $O(\log N)$, przy całkowitym wykluczeniu ryzyka kolizji dzięki opieraniu rzeczywistych identyfikatorów na standardzie UUID.

Istotnym aspektem translacji w środowisku *schema-less* jest obsługa porównań liczbowych. Choć formalna specyfikacja PQL zakłada ścisłe typowanie, rzeczywiste dzienniki zdarzeń XES często zawierają niespójności (tzw. *dirty data*), gdzie wartości liczbowe są błędnie eksportowane jako łańcuchy znaków. W CouchDB typy te są rozróżniane, przez co ścisłe zapytanie o wartość

liczbową 42 pominęłoby dokumenty z wartością „42”. Aby uodpornić system na tego typu błędy bez łamania reguł ewaluacji po stronie samego interpretera, translator stosuje na warstwie bazy danych mechanizm poszerzania selektorów. Dla operatora `eq` z wartością liczbową generowany jest selektor `$in` zawierający zarówno natywną postać liczbową, jak i jej tekstowy odpowiednik. Analogiczna, defensywna strategia jest stosowana dla operatorów `neq` (`$nin`) oraz `in`/`not in`. Niezależnie od warunków logicznych, każde wygenerowane zapytanie Mango zawiera w selektorze obligatoryjny warunek na pole `docType`, który ogranicza wyniki do dokumentów właściwego typu (`log`, `trace` lub `event`), zgodnie z zakresem kolekcji docelowej zapytania PQL.

## 4.3 Wykonywanie zapytań

Warstwa wykonawcza, zaimplementowana w klasie `PqlQueryExecutor`, jest najobszerniejszym i najbardziej złożonym komponentem systemu. Jej odpowiedzialnością jest koordynacja całego procesu wykonania zapytania PQL, obejmującego: wysłanie zapytania Mango do bazy, pobranie surowych dokumentów, ewaluację warunków w pamięci, rozwiązywanie odwołań międzyzakresowych, sortowanie, grupowanie, obliczanie wyrażeń arytmetycznych i funkcji agregujących, stosowanie hierarchicznych limitów i offsetów oraz formatowanie wyników.

### Wykonanie zapytania `select`

Proces wykonania zapytania odczytującego składa się z następujących kroków:

1. Analiza warunków zapytania pod kątem obecności odwołań międzyzakresowych (*cross-scope*) i rozdzielenie ich na warunki lokalne (dotyczące zakresu kolekcji) oraz warunki międzyzakresowe;
2. Jeśli obecne są warunki międzyzakresowe, wykonanie zapytania wstępnego do bazy w celu pobrania identyfikatorów dokumentów nadrzędnych spełniających te warunki, a następnie dodanie warunku filtrującego (`$in` na polu `logId` lub `traceId`) do zapytania głównego;
3. Translacja zapytania PQL na zapytanie Mango i wysłanie go do bazy CouchDB;
4. Strumieniowe pobranie dokumentów z odpowiedzi bazy i ewaluacja warunków w pamięci (filtracja wyników);
5. Pobranie dokumentów nadrzędnych (`trace`, `log`) potrzebnych do projekcji, sortowania lub grupowania atrybutów z wyższych zakresów;
6. Walidacja istnienia pól wymaganych przez `order by` i `group by` (mechanizm *fail-fast* zapobiegający przepełnieniu pamięci);
7. Obliczenie wyrażeń arytmetycznych i funkcji skalarnych, z zapisaniem wyników jako dodatkowych pól w dokumentach;
8. Sortowanie wyników w pamięci zgodnie z klauzulą `order by`, z uwzględnieniem wielopoziomowego porządku i kierunków sortowania (`asc`/`desc`);
9. Zastosowanie hierarchicznych limitów i offsetów;
10. Jeśli zapytanie zawiera `group by` lub funkcje agregujące: grupowanie wyników, obliczenie agregacji i formatowanie odpowiedzi grupowej;
11. W przeciwnym razie: wstrzyknięcie atrybutów nadrzędnych i formatowanie odpowiedzi płaskiej.

### Rozwiązywanie warunków międzyzakresowych (cross-scope)

Specyfikacja PQL pozwala na filtrowanie zdarzeń na podstawie atrybutów śladów lub logów (np. `select e:name where t:name = 'Order_1'`). W relacyjnej bazie danych operacja ta byłaby realizowana za pomocą `join`. W modelu dokumentowym CouchDB, gdzie każdy dokument jest niezależny, konieczne jest zastosowanie strategii dwuprzebiegowej. W pierwszym przebiegu system wykonuje zapytanie do bazy w zakresie nadrzędnym (np. `trace`) z warunkami dotyczącymi tego zakresu, pobiera wszystkie dokumenty nadrzędne, ewaluuje warunki w pamięci i zbiera identyfikatory (`_id`) spełniających je dokumentów. W drugim przebiegu identyfikatory te są dodawane jako warunek filtrujący (`$in` na polu `traceId` lub `logId`) do głównego zapytania w zakresie docelowym. Strategia ta zachowuje semantykę PQL i jednocześnie wykorzystuje mechanizm indeksowania CouchDB do efektywnego filtrowania.

### Ewaluacja warunków w pamięci.

Nie wszystkie warunki PQL mogą być bezpośrednio przetłumaczone na selektory Mango. Dotyczy to w szczególności warunków na polach zagnieżdżonych (np. `xes_attributes.org:resource`), warunków międzyzakresowych oraz złożonych wyrażeń logicznych. Z tego powodu system stosuje hybrydową strategię ewaluacji opartą na zasadzie *push-down optimization*. Translator generuje zapytanie Mango obejmujące możliwie wiele warunków: do bazy delegowane są wszystkie proste porównania (`SimpleCondition`) operujące na polach bezpośrednich oraz zdenormalizowanych (prefiksy `l:`, `t:` oraz `c:`), dla których istnieją bezpośrednie odpowiedniki w operatorach Mango. Z kolei wyrażenia wymagające obliczeń (arytmetyka, funkcje skalarne) oraz warunki na polach strukturalnie złożonych są pomijane w zapytaniu bazodanowym i pozostawiane do weryfikacji po stronie aplikacji. Następnie `PqlQueryExecutor` weryfikuje każdy pobrany dokument w pamięci za pomocą metody `evaluateConditionInMemory`. Metoda ta realizuje pełną rekurencyjną ewaluację drzewa warunków, obsługując wszystkie operatory PQL, w tym porównania liczbowe i tekstowe, operatory `in`/`not in`, `like` (z konwersją wzorca na wyrażenie regularne), `matches` (z pełną obsługą wyrażeń regularnych) oraz `is null`/`is not null`.

### Ekstrakcja wartości atrybutów

Centralna metoda `extractRobustValue` jest odpowiedzialna za wydobycie wartości atrybutu z dokumentu JSON, niezależnie od poziomu hierarchii i sposobu przechowywania danych. Metoda ta realizuje wieloetapową strategię wyszukiwania:

1. Sprawdzenie zdenormalizowanych pól z prefiksami zakresów (`l:`, `t:`) bezpośrednio w dokumencie;
2. Sprawdzenie pól specjalnych (`identity:id`, `activity`, `timestamp`);
3. Rozwiązywanie klasyfikatorów (pola z prefiksem `c:`) z uwzględnieniem mapowania niejawnych klasyfikatorów;
4. Nawigacja po ścieżce pól wygenerowanej przez `PqlFieldMapper`;
5. Przeszukanie kontenera atrybutów (`log_attributes` lub `xes_attributes`);
6. Bezpośrednie dopasowanie po nazwie atrybutu z uwzględnieniem aliasów standardowych.

Wieloetapowa strategia jest niezbędna ze względu na różnorodność sposobów przechowywania atrybutów w dokumentach CouchDB: niektóre atrybuty są zdenormalizowane na najwyższym

poziomie dokumentu (np. `activity`, `timestamp`), inne znajdują się w obiektach zagnieżdżonych (`xes_attributes`), a jeszcze inne są wstrzykiwane z dokumentów nadrzędnych podczas importu (`l:concept:name`, `t:concept:name`).

### Grupowanie wyników

System obsługuje trzy tryby grupowania, determinowane przez strukturę klauzuli `group by`:

- *Grupowanie płaskie* jest stosowane, gdy `group by` zawiera atrybuty bez znaczników wynoszenia (np. `group by e:name`). W tym trybie zdarzenia są grupowane według krotki (ang. *tuple*) wartości wskazanych atrybutów w ramach każdego śladu. Implementacja wykorzystuje tablicę mieszającą (*hash table*), w której kluczem jest lista obiektów (`List<Any?>`) reprezentująca wartości atrybutów o zróżnicowanych typach. Dla każdej grupy obliczane są funkcje agregujące, a wynik zawiera pole `count` z liczbą elementów w grupie oraz pole `events` z pełną kolekcją dokumentów należących do danej grupy.

- *Grupowanie wariantowe* jest stosowane, gdy `group by` zawiera atrybut z pojedynczym znacznikiem wynoszenia (np. `group by ^e:name`). W tym trybie ślady są grupowane według sekwencji wartości atrybutu zdarzeń, co odpowiada analizie wariantów procesu (*process variant analysis*). Dwa ślady należą do tej samej grupy, jeśli sekwencje wartości danego atrybutu ich zdarzeń są identyczne. Jest to jedna z kluczowych operacji eksploracji procesów, umożliwiająca identyfikację najczęstszych przebiegów procesu.

- *Grupowanie globalne niejawne* jest stosowane, gdy zapytanie zawiera funkcje agregujące, ale nie zawiera klauzuli `group by` (np. `select count(e:name), avg(e:amount)`). W tym trybie agregacje są obliczane dla całego zbioru wynikowego, jednak procesor agregujący uwzględnia zakresy (*scope*) zdefiniowane przy każdej funkcji (np. oddzielnie zliczając unikalne wystąpienia śladów i wszystkie wystąpienia zdarzeń w ramach tego samego zapytania)

### Funkcje agregujące i skalarne

System obsługuje pięć funkcji agregujących zdefiniowanych w specyfikacji PQL: `count`, `sum`, `avg`, `min` i `max`. Funkcje te operują na wartościach wydobytych z dokumentów w grupie za pomocą metody `extractRobustValue`. Dla atrybutów zawierających znaczniki czasu wartości tekstowe są parsowane do postaci numerycznej (milisekundy od epoki Unix) z wykorzystaniem współbieżnej pamięci podręcznej (`ConcurrentHashMap`), co znacząco przyspiesza wielokrotne parsowanie tych samych dat. Zgodnie ze specyfikacją PQL, funkcja `count` zwraca liczbę niepustych (*non-null*) wartości atrybutu w grupie. Aby jednak zapewnić poprawność analityczną w warunkach denormalizacji danych (gdzie atrybuty nadrzędne są powielone w dokumentach zdarzeń), implementacja w zakresach `log` i `trace` zlicza wartości unikalne. Zapobiega to błędnemu utożsamianiu liczby zdarzeń z liczbą encji nadrzędnych. Funkcje skalarne jednoargumentowe (`year`, `month`, `day`, `hour`, `minute`, `second`, `millisecond`, `quarter`, `dayofweek`, `upper`, `lower`, `round`, `date`, `time`) operują na wartości pojedynczego wyrażenia. Funkcja `now` zwraca bieżący znacznik czasu. Wyrażenia arytmetyczne są ewaluowane rekurencyjnie za pomocą metody `evaluateMath`, która obsługuje cztery operatory (`+`, `-`, `*`, `/`) oraz wewnętrzną obsługę typów numerycznych. Mechanizm ten ujednolica reprezentację danych pochodzących z dokumentów JSON, co pozwala na poprawne wykonanie operacji matematycznych bez wprowadzania jawnych operatorów rzutowania do składni języka. Szczególną obsługą objęta jest różnica dat: odejmowanie dwóch wartości tekstowych w formacie ISO 8601 jest interpretowane jako obliczenie różnicy w dniach.

### Hierarchiczne limity i offsety

Specyfikacja PQL umożliwia definiowanie limitów i offsetów na poszczególnych poziomach hierarchii (np. `limit l:1, t:3, e:5`). System realizuje tę funkcjonalność za pomocą metody `applyHierarchicalLimitsAndOffsets`, która przetwarza dokumenty sekwencyjnie, utrzymując liczniki dla każdego poziomu hierarchii. Algorytm identyfikuje przynależność każdego dokumentu do logu i śladu (na podstawie pól `logId` i `traceId`), sprawdza, czy aktualny log, ślad i zdarzenie mieszczą się w dozwolonych zakresach, i przepuszcza lub odrzuca dokument. Po zastosowaniu limitów hierarchicznych stosowany jest jeszcze globalny limit i offset.

### Wykonywanie zapytań `delete` z kaskadowaniem

Zapytania usuwające są realizowane przez metodę `executeDelete`, która respektuje hierarchiczną strukturę danych XES zgodnie z normą IEEE 1849-2016. Usunięcie logu powoduje kaskadowe usunięcie wszystkich jego śladów i zdarzeń, a usunięcie śladu powoduje usunięcie wszystkich jego zdarzeń. Procedura `delete` składa się z następujących etapów:

1. pobranie dokumentów spełniających warunki zapytania (z obsługą warunków międzyzakresowych i ewaluacją w pamięci);
2. zastosowanie sortowania, offsetu i limitu w pamięci;
3. zebranie identyfikatorów dokumentów do usunięcia;
4. pobranie dokumentów potomnych (kaskadowanie);
5. usunięcie wszystkich zebranych dokumentów za pomocą masowej operacji $_bulk_docs$.

### Strumieniowy importer XES

Komponent `LogImporter` odpowiada za wczytywanie dzienników zdarzeń w formacie XES do bazy CouchDB. Importer obsługuje trzy formaty wejściowe: pliki XES (nieskompresowane), pliki skompresowane GZIP (`.xes.gz`) oraz archiwa ZIP zawierające pliki XES. Detekcja formatu odbywa się na podstawie rozszerzenia pliku (klasa `LogFileDetector`). Parsowanie pliku XES jest realizowane przez klasę `StaxXesParser`, która implementuje interfejs `Iterator` i wykorzystuje strumieniowy parser XML typu StAX (*Streaming API for XML*). W odróżnieniu od parserów DOM, które wczytują cały dokument do pamięci, StAX przetwarza dokument sekwencyjnie, emitując poszczególne elementy (`XESLogAttributes`, `XESTrace`) w miarę ich napotkania w strumieniu XML. Dzięki temu importer może przetwarzać pliki o dowolnym rozmiarze bez ryzyka przepełnienia pamięci. Atrybuty zagnieżdżone są spłaszczane rekurencyjnie do postaci par klucz-wartość z kluczami złożonymi (np. `meta_concept:named_events_total:haptoglobine`), co zachowuje informację o hierarchii bez konieczności przechowywania drzewiastej struktury. Dodatkowo, parser i mapper wspólnie generują techniczny obiekt `_types`, który utrwala oryginalne mapowanie typów (np. `id`, `date`) dla każdego atrybutu, zabezpieczając system przed utratą semantyki. Klasa `StreamingXesToCouchDBMapper` realizuje mapowanie sparsowanych elementów XES na dokumenty JSON i ich zapis do bazy CouchDB. Mapper operuje w trybie wsadowym (*batch mode*): dokumenty są serializowane do bufora JSON (`ByteArrayOutputStream`) po 2500 sztuk, po czym paczka jest wysyłana do bazy za pomocą operacji `_bulk_docs`. Serializacja odbywa się bezpośrednio przez `JsonWriter` (biblioteka Gson) z pominięciem tworzenia pośrednich obiektów `JsonObject`, co minimalizuje zużycie pamięci. Zapis paczek jest zrównoleglony za pomocą puli

wątków (`ThreadPoolExecutor`) o konfiguralnym stopniu równoległości (domyślnie 4 wątki), z polityką `CallerRunsPolicy` zapobiegającą przepełnieniu kolejki. Po zakończeniu importu system automatycznie tworzy indeksy B-tree na polach `docType`, `logId`, `traceId`, `activity`, `timestamp` oraz `identity:id`, co jest niezbędne dla wydajnego wykonywania zapytań PQL.

### Eksporter CouchDB do XES

Klasa `XesExporter` realizuje proces odwrotny do importu: na podstawie identyfikatora logu pobiera z bazy dokumenty logu, śladów i zdarzeń, odtwarza hierarchiczną strukturę log-trace-event i generuje plik XML zgodny ze standardem XES. Eksporter pobiera dokumenty w trzech zapytaniach (po jednym dla każdego typu), grupuje zdarzenia według `traceId`, a następnie sortuje je ściśle według zapisanego w bazie pola `eventIndex`. Gwarantuje to bezbłędne odtworzenie oryginalnej sekwencji zdarzeń (nawet dla identycznych znaczników czasu – tzw. *tied timestamps*). Generowanie wynikowego pliku XML uwzględnia poprawne typy standardu XES (`string`, `date`, `int`, `float`, `boolean`, `id`). Identyfikacja oryginalnego typu opiera się w pierwszej kolejności na technicznym obiekcie `_types`, wstrzykniętym podczas importu, co pozwala na w pełni bezstratną rekonstrukcję (*lossless round-trip*). W przypadku braku jawnej deklaracji w `_types`, system stosuje mechanizm wnioskowania (*fallback*) na podstawie analizy struktury wartości (np. format ISO 8601, obecność kropki dziesiętnej). Atrybuty złożone (np. kontener `a|b[c]`) są rekurencyjnie składane z powrotem w listy i kontenery, a wartości specjalne w tekście XML podlegają operacji *escaping* (`&`, `<`, `>`, `"`, `'`).

### Interfejs REST API

System udostępnia interfejs programistyczny w postaci czterech endpointów REST, zaimplementowanych w klasie `PqlRestController` za pomocą frameworka Spring Boot:

- `POST /api/pql/query` przyjmuje zapytanie PQL w postaci tekstu (`Content-Type: text/plain`), wykonuje je za pośrednictwem klasy `PqlInterpreter` i zwraca wynik w formacie JSON;
- `POST /api/pql/import` przyjmuje plik XES (`multipart/form-data`), zapisuje go tymczasowo, uruchamia importer i zwraca identyfikator wgranego logu;
- `GET /api/pql/export/{logId}` eksportuje wskazany log do pliku XES i zwraca go jako załącznik do pobrania (`Content-Disposition: attachment`);
- `DELETE /api/pql/deleteAll` czyści całą bazę danych, usuwając i ponownie tworząc kolekcję.

Komunikacja z bazą CouchDB jest realizowana przez klasę `CouchDBManager`, która enkapsuluje klienta HTTP (OkHttp3) z pulą połączeń (`ConnectionPool`), konfigurowalnymi limitami równoczesnych żądań oraz uwierzytelnianiem HTTP Basic. Klasa ta udostępnia metody do tworzenia i usuwania baz, wstawiania dokumentów wsadowo (`_bulk_docs`), wyszukiwania dokumentów (`_find`) w trybie strumieniowym, tworzenia indeksów (`_index`) oraz usuwania dokumentów.

### Gwarancje transakcyjne i spójność danych

Zastosowanie strategii dwuprzebiegowej oraz rozproszenie logiki między translator a ewaluator w pamięci nakłada określone uwarunkowania na spójność danych. W odróżnieniu od systemów

relacyjnych, CouchDB nie oferuje mechanizmu blokad ani transakcji wielodokumentowych. Gwarantuje ona atomowość i izolację wyłącznie na poziomie pojedynczego dokumentu (model BASE) [9].

W kontekście interpretera PQL oznacza to brak pełnej izolacji typu *Snapshot Isolation* dla zapytań międzyzakresowych. Istnieje teoretyczne ryzyko wystąpienia anomalii odczytu, jeśli struktura logu uległaby zmianie w interwale milisekundowym pomiędzy fazami zapytania. Decyzja o akceptacji tego modelu spójności wynika ze specyfiki domeny *Process Mining*. Dzienniki zdarzeń są z zasady danymi historycznymi i niemutowalnymi (*append-only*). Raz zaimportowany log nie podlega edycji atrybutów strukturalnych, co czyni dwuprzebiegowy odczyt w pełni bezpiecznym analitycznie. Ponadto rezygnacja z globalnych blokad pozwala na zachowanie wysokiej wydajności przy przetwarzaniu milionów zdarzeń, co jest priorytetem opracowanego systemu.

## 4.4 Sposoby zapewnienia wydajności

Wydajność systemu interpretera PQL jest determinowana przez kilka czynników: czas odpowiedzi bazy CouchDB, koszt przetwarzania wyników w pamięci oraz narzut komunikacji sieciowej. W niniejszej sekcji opisano mechanizmy optymalizacyjne zastosowane w systemie w celu minimalizacji wpływu tych czynników.

### Indeksowanie

Apache CouchDB wykorzystuje indeksy typu B-tree do przyspieszenia operacji wyszukiwania na endpoincie `_find`. Bez odpowiednich indeksów każde zapytanie wymaga pełnego przeglądu kolekcji (*full collection scan*), co jest nieakceptowalne dla dużych zbiorów danych. System automatycznie tworzy indeksy na sześciu polach zidentyfikowanych jako krytyczne dla zachowania wydajności silnika oraz spójności hierarchicznego modelu danych. Pole `docType` jest niezbędne dla każdego zapytania w celu natywnej separacji typów dokumentów w ramach jednej kolekcji. Pola `logId` oraz `traceId` stanowią fundament relacji nadrzędnych, umożliwiając sprawne wykonywanie zapytań międzyzakresowych oraz operacji kaskadowego usuwania. Indeksowanie atrybutów `activity` oraz `timestamp` wynika bezpośrednio z ich roli jako obligatoryjnych wymiarów analitycznych w standardzie IEEE 1849-2016, podczas gdy indeks na `identity:id` zapewnia wydajność wyszukiwania punktowego. Tworzenie indeksu jest idempotentne: system utrzymuje zbiór sygnatur już utworzonych indeksów i pomija duplikaty.

### Strumieniowe przetwarzanie danych

System konsekwentnie stosuje podejście strumieniowe w trzech kluczowych miejscach przetwarzania. Po pierwsze, parser XML (`StaxXesParser`) wykorzystuje interfejs StAX, przetwarzając plik XES sekwencyjnie bez wczytywania go w całości do pamięci. Po drugie, mapper (`StreamingXesToCouchDBMapper`) serializuje dokumenty JSON bezpośrednio do strumienia bajtów (`ByteArrayOutputStream`) za pomocą `JsonWriter`, unikając tworzenia pośrednich drzew obiektów. Po trzecie, metoda `findDocsStream` w `CouchDBManager` parsuje odpowiedź JSON z bazy za pomocą strumieniowego czytnika Gson (`JsonReader`), wywołując *callback* dla każdego dokumentu zamiast budować tablicę wyników w pamięci. Takie podejście zapewnia stałe zużycie pamięci niezależnie od rozmiaru przetwarzanych danych, co jest szczególnie istotne przy importowaniu dzienników zawierających miliony zdarzeń.

### Wsadowy zapis z równoległością

Importer XES zapisuje dokumenty do CouchDB w paczkach po 2500 dokumentów za pomocą operacji `_bulk_docs`, która jest znacząco wydajniejsza niż pojedyncze operacje wstawiania. Paczki są wysyłane równolegle za pomocą puli wątków (`ThreadPoolExecutor`) z konfiguralnym stopniem równoległości (domyślnie 6 wątków). Polityka `CallerRunsPolicy` zapewnia przeciwciśnienie (*backpressure*): gdy kolejka puli jest pełna, wątek producenta sam wykonuje zapis, co zapobiega nadmiernemu buforowaniu w pamięci. Klient HTTP (OkHttp3) jest skonfigurowany z pulą połączeń (`ConnectionPool`) o rozmiarze odpowiadającym stopniowi równoległości, z limitami czasu operacji dostosowanymi do dużych paczek (120 sekund na zapis).

### Denormalizacja danych.

Kluczową optymalizacją jest wstrzykiwanie atrybutów z wyższych zakresów (log, trace) bezpośrednio do dokumentów zdarzeń podczas importu. Pola `l:concept:name` i `t:concept:name`, a także wartości klasyfikatorów (pola `c:*`), są dodawane do każdego dokumentu zdarzenia. Dzięki temu najczęstsze zapytania PQL filtrujące zdarzenia po nazwie śladu lub logu (np. `where t:name = 'Order_1'`) mogą być realizowane bez dodatkowych zapytań do bazy. Kosztem tej optymalizacji jest zwiększony rozmiar dokumentów i redundancja danych, jednak jest to świadomy kompromis typowy dla baz dokumentowych, gdzie priorytetem jest szybkość odczytu.

### Pamięci podręczne.

System wykorzystuje dedykowane struktury w pamięci: współbieżną pamięć podręczną parsowania dat (`dateParsingCache`) opartą na `ConcurrentHashMap` (...) oraz zbiór sygnatur indeksów (`ensuredIndexes`) zapobiegający wielokrotnemu tworzeniu tego samego indeksu na poziomie menedżera bazy. Dodatkowo metoda `extractRobustValue` wykorzystuje internalizację łańcuchów (`String.intern()`) dla często powtarzających się wartości, co redukuje zużycie pamięci w przypadku dużych zbiorów danych z ograniczonym słownikiem wartości.

### Złożoność obliczeniowa

Złożoność obliczeniowa poszczególnych operacji systemu zależy od struktury zapytania i rozmiaru danych. Poniższa tabela podsumowuje złożoność głównych operacji, gdzie $N$ oznacza liczbę dokumentów w kolekcji docelowej, $M$ liczbę dokumentów nadrzędnych, a $K$ liczbę grup:

| **Operacja** | **Złożoność** | **Uwagi** |
|---|---|---|
| Zapytanie bez `order by` / `group by` | $O(N)$ | Pojedynczy przebieg z filtracją strumieniową |
| Zapytanie z `order by` | $O(N \log N)$ | Sortowanie w pamięci (Timsort) |
| Zapytanie z warunkami *cross-scope* | $O(M + N)$ | Dwa przebiegi: nadrzędny + docelowy |
| Grupowanie płaskie | $O(N + K \log K)$ | Budowa map + opcjonalne sortowanie grup |
| Grupowanie wariantowe | $O(N \log N + K \log K)$ | Sortowanie zdarzeń w śladach + sortowanie wariantów |
| Agregacje (`count`, `sum`, `avg`, `min`, `max`) | $O(N)$ | Pojedynczy przebieg po elementach grupy |
| Hierarchiczne `limit` / `offset` | $O(N)$ | Pojedynczy przebieg z licznikami poziomów |
| Import XES (wsadowy) | $O(N)$ | Liniowy względem liczby elementów XES |
| `delete` z kaskadowaniem | $O(N + C)$ | $C$ = liczba dokumentów potomnych |

Tabela 4.3: Złożoność obliczeniowa operacji systemu

Należy zauważyć, że operacje przetwarzania w pamięci (sortowanie, grupowanie, ewaluacja warunków) są ograniczone przez konfigurowalny limit dokumentów pobieranych z bazy (domyślnie 1 000 000). Limit ten stanowi kompromis między kompletnością wyników a bezpieczeństwem zasobów serwera (zapobieganie błędom `OutOfMemoryError`). W przypadku przekroczenia tego progu, system przyjmuje defensywną strategię *fail-fast*: egzekucja zapytania zostaje przerwana, a użytkownik otrzymuje błąd wraz z informacją o przekroczeniu dopuszczalnego wolumenu danych. Takie podejście eliminuje ryzyko interpretacji niekompletnych wyników jako poprawnych, co mogłoby prowadzić do błędnej analizy procesów. Dla zapytań, w których CouchDB może bezpośrednio zastosować selektor Mango i limity, złożoność jest determinowana wyłącznie przez liczbę zwróconych dokumentów, a nie przez rozmiar całej kolekcji.

Wąskim gardłem wydajnościowym systemu jest komunikacja sieciowa z bazą CouchDB, realizowana za pośrednictwem protokołu HTTP. Każde zapytanie wymaga co najmniej jednego żądania HTTP do endpointu `_find`, a zapytania z warunkami międzyzakresowymi wymagają dodatkowych żądań. Narzut ten wynika nie tylko z samych opóźnień sieciowych (*latency*), ale przede wszystkim z wysokiego kosztu procesora związanego z deserializacją masywnych dokumentów JSON na obiekty języka Kotlin. Ponieważ CouchDB nie wspiera protokołów binarnych, narzut tekstowego formatu danych staje się dominującym czynnikiem przy dużych zbiorach wynikowych. Opóźnienia te są minimalizowane przez zastosowanie puli połączeń (*connection pooling*) i utrzymywanie trwałych połączeń HTTP (*keep-alive*).

# Rozdział 5

# Eksperymenty

W tym rozdziale przedstawiono opis przeprowadzonych eksperymentów, których celem była weryfikacja poprawności semantycznej, integralności danych oraz wydajności i skalowalności opracowanego interpretera PQL działającego w oparciu o środowisko NoSQL.

## 5.1 Pytania eksperymentalne

Proces badawczy został sformułowany wokół czterech kluczowych pytań, które weryfikują architekturę systemu pod kątem gotowości do pracy z rzeczywistymi dziennikami zdarzeń (*event logs*).

### Zgodność wyników ze specyfikacją PQL

Czy wyniki zwracane przez interpreter PQL są zgodne ze specyfikacją PQL [25] Istotnym aspektem badań jest wykazanie, że migracja z relacyjnej bazy SQL na nierelacyjną bazę dokumentową (NoSQL) nie wpłynęła negatywnie na poprawność analityczną. Weryfikacja opiera się na dwuetapowym procesie: regresyjnych testach jednostkowych (`PqlInterpreterTest`) przeniesionych z implementacji referencyjnej oraz wykonaniu ponad 30 nowych, rygorystycznych scenariuszy integracyjnych (`PqlEvaluationTest`). Scenariusze te sprawdzają zachowanie systemu w obliczu skomplikowanych konstrukcji, takich jak wielowymiarowe grupowanie z hoistingiem, filtrowanie *cross-scope*, dynamiczne rozwiązywanie klasyfikatorów oraz sortowanie *Nulls Last*. Celem eksperymentu jest udowodnienie, że autorski silnik zapytań zachowuje pełną spójność semantyczną z formalną specyfikacją języka PQL [25].

### Niezmienność danych (Lossless Round-trip)

Czy interpreter gwarantuje niezmienność danych? Eksperyment ten weryfikuje, czy każdy dziennik zdarzeń załadowany do bazy dokumentowej można zrekonstruować i wyeksportować z powrotem do pliku XML w niezmienionej formie. Jest to właściwość dowodząca, że dwukierunkowa transformacja między hierarchicznym standardem XES [3] a strukturą obiektów JSON przebiega całkowicie bezstratnie (*Lossless Round-trip*). Testy sprawdzają zachowanie nie tylko standardowych atrybutów, ale również głęboką semantykę dziennika. Obejmuje to poprawność rzutowania natywnych typów danych (np. wartości logicznych czy dat), nienaruszalność pierwotnych identyfikatorów biznesowych oraz zachowanie nagłówków globalnych i rozszerzeń. Walidacja opiera się na weryfikacji równoważności semantycznej (*Semantic Equivalence Check*), wykluczając błędy wynikające np. ze zmiany kolejności węzłów w drzewie dokumentu.

### Skalowalność względem rozmiaru danych

Jak skaluje się interpreter względem rozmiaru danych wejściowych? Celem eksperymentu jest zbadanie czasu wykonania 8 zróżnicowanych przypadków użycia (obejmujących różnorodne mechanizmy PQL, takie jak projekcje, filtrowanie *cross-scope*, wyrażenia regularne, agregacje oraz grupowanie wariantów) w zależności od dwóch czynników: całkowitej liczby zdarzeń załadowanych do bazy oraz rozmiaru zbioru wynikowego zwracanego przez zapytanie (wymuszanego klauzulą *limit* na poziomie od 10 do 10 000 rekordów). Ze względu na uwarunkowania sprzętowe maszyny testowej, eksperymenty zaplanowano dla zbiorów danych o wielkości od około $2 \cdot 10^3$ do ponad $10^6$ zdarzeń. Aby zapewnić najwyższą obiektywność analizy, do pomiarów wykorzystany zostanie czas procesora (*CPU time*), co pozwala wyizolować czysty narzut obliczeniowy interpretera od opóźnień sieciowych i dyskowych ukrytych w całkowitym czasie rzeczywistym (*Real time*).

### Narzut pamięciowy optymalizacji architektonicznych

Jak denormalizacja atrybutów z wyższych poziomów (log, ślad) do poziomu zdarzenia wpływa na rozmiar bazy danych? Zastosowanie modelu NoSQL wymaga unikania kosztownych złączeń `join`, co zmusza do redundancji danych. Celem tego eksperymentu jest precyzyjne zmierzenie kosztu pamięciowego (*storage overhead*) zastosowanej optymalizacji i odpowiedź na pytanie, czy kompromis między nadmiarowym użyciem przestrzeni dyskowej a wydajnością odczytu jest opłacalny.

## 5.2 Środowisko eksperymentalne

Aby zagwarantować wiarygodność i powtarzalność wyników, wszystkie eksperymenty przeprowadzono w kontrolowanym środowisku sprzętowo-programowym. Testy wykonano na komputerze o parametrach odpowiadających typowej stacji roboczej. Dodatkowo, aby zapobiec zakłóceniom podczas pomiarów, ograniczono do minimum aktywność procesów i aplikacji działających w tle.

### Specyfikacja sprzętowa i programowa

- **Procesor:** Intel Core i5-12450H (8 rdzeni, 12 wątków).
- **Pamięć RAM:** 16 GB DDR4 (taktowanie 3200 MHz).
- **Dysk:** SSD NVMe WD Blue SN580 1TB (interfejs PCIe Gen4, odczyt do 4150 MB/s).
- **System operacyjny:** Microsoft Windows 11 Education (wersja 10.0.26200, 64-bit).
- **Baza danych:** Apache CouchDB 3.3.3 (konfiguracja jednowęzłowa) [9].
- **Środowisko uruchomieniowe:** OpenJDK 21 (Temurin-21.0.6) z limitem pamięci `-Xmx8G`.

### Przygotowanie danych testowych

W celu przeprowadzenia weryfikacji funkcjonalnej wykorzystano dedykowane dzienniki zdarzeń XES, generowane dynamicznie w trakcie wykonywania testów. Zbiory te zostały precyzyjnie zaprojektowane tak, aby wymusić pokrycie wszystkich obsługiwanych typów atrybutów (w tym `string, int, float, boolean, date, id`) oraz skomplikowanych struktur standardu, takich jak rozszerzenia, atrybuty globalne, zagnieżdżenia czy definicje klasyfikatorów. Do ewaluacji analitycznej oraz testów wydajnościowych (skalowalności) zastosowano logi benchmarkowe, m.in. *JournalReview-extra.xes* oraz *Hospital_log.xes.gz* (stanowiący reprezentację rzeczywistych danych z konkursu BPI

Challenge [31]). Ponadto, w celu zbadania granic wydajnościowych systemu przy skrajnych obciążeniach, log szpitalny zaimportowano do bazy danych wielokrotnie, co pozwoliło na uzyskanie przestrzeni testowej przekraczającej wolumen miliona zdarzeń.

### Metodyka pomiarowa

Czas wykonania zapytań rejestrowany jest dwutorowo, co pozwala na kompleksową ocenę wydajności systemu. Pierwszą metrykę stanowi CPU Time, mierzony za pomocą interfejsu `java.lang.management.ThreadMXBean`. Podejście to pozwala wyizolować czysty narzut obliczeniowy interpretera, wykluczając czas oczekiwania na odpowiedzi z bazy danych oraz zakłócenia wynikające z pracy harmonogramu zadań systemu operacyjnego. Drugą metryką jest Real Time (czas zegarowy), odzwierciedlający sumaryczny czas oczekiwania użytkownika na wynik, uwzględniający operacje wejścia/wyjścia (I/O) oraz komunikację sieciową z serwerem CouchDB.

## 5.3 Zgodność wyników ze specyfikacją PQL

Czy wyniki zwracane przez interpreter PQL są zgodne ze specyfikacją PQL?

### Przebieg procedury testowej

Proces weryfikacji zrealizowano przy użyciu dwóch uzupełniających się pakietów testowych, które poddano rygorystycznej automatyzacji:

1. **Walidacja zgodności wstecznej (`PqlInterpreterTest`):** Wykorzystano zestaw scenariuszy oparty na oryginalnych testach referencyjnych dla języka PQL [24]. Na tym etapie sprawdzono poprawność obsługi fundamentów języka: operatorów porównania, logiki trójwartościowej (uwzględniającej `null`), sortowania *Nulls Last* oraz specyficznego dla standardu XES rzutowania typów (`boolean`, `date`, `float`).

2. **Ewaluacja analityczna (`PqlEvaluationTest`):** Przeprowadzono testy końcowe (E2E) składające się z 34 kompleksowych zapytań. Wykorzystano w nich zarówno logi syntetyczne (do weryfikacji przypadków brzegowych), jak i rzeczywiste dzienniki benchmarkowe. Testy te wymusiły sprawdzenie najbardziej zaawansowanych mechanizmów silnika, takich jak dynamiczne klejenie klasyfikatorów w locie oraz wielopoziomowe limity (*hierarchical limits*).

### Analiza wyników i spójności danych

Eksperymenty potwierdziły pełną zgodność semantyczną silnika, którą zweryfikowano dwutorowo. W pierwszej kolejności przeprowadzono walidację zgodności wstecznej (`PqlInterpreterTest`). Na tym etapie potwierdzono poprawność obsługi fundamentów języka. Następnie przeprowadzono właściwe testy ewaluacyjne na bardziej złożonych strukturach. Wszystkie 34 zapytania zakończyły się sukcesem, a zwrócone wartości (poza formą prezentacji) były identyczne z oczekiwanymi. Podsumowanie skuteczności w podziale na kluczowe obszary funkcjonalne przedstawia Tabela 5.1.

| Kategoria operacji PQL | Liczba testów | Wynik |
|---|---|---|
| Filtrowanie (w tym: *cross-scope*, `regex`, `not in`) | 10 | 10 / 10 |
| Matematyka i funkcje skalarne (w tym operacje na datach ISO-8601) | 9 | 9 / 9 |
| Grupowanie sekwencyjne (*Variant Mining*) i agregacje | 10 | 10 / 10 |
| Transformacja logu (`delete`) oraz sortowanie hierarchiczne | 5 | 5 / 5 |

Tabela 5.1: Wyniki integracyjnej ewaluacji semantycznej.

### Kluczowe przypadki techniczne

W trakcie analizy wyników szczególną uwagę poświęcono mechanizmom, które w bazach NoSQL stanowią największe wyzwanie implementacyjne:

- **Dynamiczna obsługa klasyfikatorów:** System poprawnie obsłużył zapytania o klasyfikatory złożone (np. `[concept:name+lifecycle:transition]`), mimo że nie występowały one jako gotowe klucze w bazie CouchDB. Silnik automatycznie zrekonstruował te wartości w pamięci RAM, co umożliwiło poprawną analizę wariantów procesu.
- **Poprawność mechanizmu hoisting:** Dzięki precyzyjnemu oczyszczaniu prefiksów (`^` oraz `[`), system bezbłędnie zidentyfikował wszystkie 97 unikalnych wariantów w logu *Journal-Review*, unikając spłaszczania grup przy zapytaniach o atrybuty zdarzeń przenoszone do poziomu śladu.
- **Integralność po operacjach DELETE:** Weryfikacja po wykonaniu zapytań usuwających wykazała, że system precyzyjnie usuwa rekordy zdarzeń przy zachowaniu pełnej integralności dokumentów nadrzędnych (Log i Trace) oraz ich metadanych.

### Struktura wyników (JSON vs Table)

Należy podkreślić świadomą różnicę w formacie wyjściowym. Podczas gdy systemy SQL-owe zwracają płaskie wiersze, opracowany silnik generuje hierarchiczny format JSON. W przypadku grupowania, wynik zawiera tablicę `"events"`, co pozwala na natychmiastowe renderowanie wariantów w interfejsie użytkownika bez konieczności wykonywania dodatkowych zapytań o szczegóły zdarzeń.

## 5.4 Niezmienność danych (Lossless Round-trip)

Czy interpreter gwarantuje niezmienność danych?

### Przebieg procedury testowej

Zdolność systemu do bezstratnego przechowywania i rekonstrukcji informacji zweryfikowano w ramach pełnego cyklu życia danych (*Round-trip*). Proces weryfikacji zrealizowano przy użyciu pakietu `XesExporterTest`, który obejmował pięć zróżnicowanych scenariuszy badawczych:

1. **Weryfikacja tożsamości semantycznej (*Deep Semantic Identity*):** Do bazy zaimportowano złożony, syntetyczny dziennik zawierający pełne spektrum struktur standardu XES (rozszerzenia, atrybuty globalne, klasyfikatory) oraz zagnieżdżone kontenery (`list`, `container`). Po eksporcie sprawdzono programowo zgodność każdego atrybutu i typu danych.

2. **Test skali i kompletności (*Round-trip Test*):** Wykorzystano log benchmarkowy *JournalReview*, weryfikując, czy po eksporcie liczba śladów (101) oraz zdarzeń (2298) pozostaje nienaruszona, a wszystkie relacje rodzic-dziecko są zachowane.

3. **Integralność po mutacji (*Mutation Test*):** Sprawdzono zachowanie struktury logu po wykonaniu operacji `delete`. Test potwierdził, że selektywne usuwanie zdarzeń nie uszkadza metadanych logu oraz atrybutów pozostałych śladów i eventów.

4. **Zachowanie porządku zdarzeń (*Event Order Preservation*):** Zweryfikowano zachowanie systemu w przypadku tzw. problemu identycznych znaczników czasu (*tied timestamps*). Test dowiódł, że dzięki zastosowaniu technicznego indeksowania, eksporter bezbłędnie zachowuje oryginalną sekwencję zdarzeń wewnątrz śladu, niezależnie od rozdzielczości czasowej logu.

5. **Odporność na obciążenie (*OOM Scale Test*):** Przeprowadzono eksport masywnego dziennika zdarzeń (*Hospital_log*), aby dowieść, że zaimplementowany mechanizm strumieniowania danych zapobiega występowaniu błędów braku pamięci (`OutOfMemoryError`).

### Analiza wyników i spójności danych

Eksperymenty potwierdziły **pełną niezmienność semantyczną** danych. System pomyślnie przeszedł wszystkie testy regresyjne, dowodząc, że transformacja z hierarchicznego formatu XML do dokumentowego formatu JSON jest operacją całkowicie bezstratną. Podsumowanie wyników weryfikacji przedstawia Tabela 5.2.

| Scenariusz weryfikacyjny | Liczba prób | Wynik |
|---|---|---|
| Zachowanie typów danych i struktur zagnieżdżonych | 1 | Pozytywny |
| Zgodność wolumenu danych (ślady i zdarzenia) | 1 | Pozytywny |
| Integralność struktur po operacjach modyfikujących | 1 | Pozytywny |
| Bezbłędne zachowanie oryginalnej sekwencji zdarzeń | 1 | Pozytywny |
| Stabilność eksportu przy dużym wolumenie danych | 1 | Pozytywny |

Tabela 5.2: Wyniki testów niezmienności danych w cyklu Round-trip.

### Kluczowe przypadki techniczne

Podczas implementacji eksportera rozwiązano kluczowe problemy techniczne wynikające ze specyfiki baz dokumentowych:

- **Mechanizm rekonstrukcji typów (`_types`):** Dzięki wprowadzeniu technicznego pola `_types`, silnik poprawnie odróżnia natywne typy XES (np. `id`, `date`) od standardowych typów JSON. Pozwala to na uniknięcie błędów rzutowania przy ponownym generowaniu tagów XML.

- **Odwzorowanie struktur złożonych:** Zastosowano algorytm rekurencyjny, który na podstawie spłaszczonych kluczy w bazie NoSQL odtwarza oryginalną strukturę zagnieżdżonych list i kontenerów, zachowując ich hierarchiczną przynależność.

- **Separacja identyfikatorów:** System skutecznie oddziela wewnętrzne techniczne identyfikatory bazy CouchDB (`_id`) od biznesowych identyfikatorów logu (`identity:id`), co gwarantuje, że wyeksportowany dziennik jest identyczny z oryginałem pod kątem biznesowym.

- **Gwarancja chronologii (`eventIndex`):** Aby rozwiązać problem niedeterministycznego sortowania bazy NoSQL dla zdarzeń o identycznym czasie wystąpienia, podczas importu wstrzykiwane jest pole `eventIndex`. Eksporter wykorzystuje je do ścisłej rekonstrukcji oryginalnej listy zdarzeń wewnątrz śladu, co jest kluczowe z punktu widzenia poprawnego odkrywania modeli procesów.

### Wyjaśnienie rozbieżności technicznych i zachowanie porządku zdarzeń

Należy odnotować, że jedyną zaobserwowaną różnicą strukturalną pomiędzy plikiem wejściowym a wyjściowym XES była zmieniona kolejność występowania atrybutów wewnątrz poszczególnych węzłów. Wynika to z faktu, że standard JSON przechowuje obiekty jako mapy nieuporządkowane (*unordered maps*). Zgodnie z definicją formalną standardu XES, atrybuty na danym poziomie hierarchii stanowią zbiór nieuporządkowany (*unordered set*), zatem ich kolejność w dokumencie XML nie posiada najmniejszego znaczenia semantycznego.

Z kolei w odniesieniu do sekwencji zdarzeń, standard XES rygorystycznie wymaga zachowania ich oryginalnego porządku (*ordered list*), ponieważ determinuje on rzeczywisty przebieg wariantów procesu. Dzięki wspomnianemu wyżej wdrożeniu pola `eventIndex`, eksporter nie polega na niedeterministycznym sortowaniu po znacznikach czasu (*timestamps*), lecz precyzyjnie rekonstruuje natywną sekwencję wykonania. Tym samym udowodniono, że opracowany system zapewnia stuprocentową poprawność merytoryczną i analityczną danych wyjściowych w procesie dwukierunkowej transformacji.

## 5.5 Skalowalność względem rozmiaru danych wejściowych

Jak skaluje się interpreter względem rozmiaru danych wejściowych i zwracanych?

### Przebieg procedury testowej

Aby określić charakterystykę wydajnościową i wskazać moment krytyczny (degradację wydajności), zdefiniowano 8 zróżnicowanych przypadków użycia. Odzwierciedlają one operacje typowe dla dziedziny *Process Mining*:

1. **Q1 (Projekcja):** Pobranie pojedynczego atrybutu zdarzenia (`select e:concept:name`).
2. **Q2 (Filtracja 0-CPU):** Zapytanie z klauzulą `where` celowo zaprojektowane tak, aby baza odrzuciła 100% rekordów przed wysłaniem ich do aplikacji (`where e:concept:name = 'NON_EXISTENT_EVENT'`).
3. **Q3 (Regex):** Kosztowne skanowanie tekstowe na poziomie bazy za pomocą wyrażeń regularnych (`where e:concept:name matches '.*(A_).*'`).
4. **Q4 (Cross-scope):** Filtracja zdarzeń na podstawie nadrzędnego atrybutu śladu (`where t:concept:name = 'Trace_1'`).
5. **Q5 (Agregacja globalna):** Obliczenie wartości skrajnych i ilościowych dla całego dziennika zdarzeń (`select count(e:concept:name), min(e:time:timestamp)`).
6. **Q6 (Sortowanie):** Test obciążenia pamięci RAM przez klauzulę wymuszającą globalne sortowanie dokumentów (`order by e:time:timestamp desc`).

7. **Q7 (Płaskie Grupowanie):** Klasyczna agregacja wymiarowa po atrybucie zdarzenia z równoległym przeliczaniem funkcji agregującej (`select e:concept:name, count(e:time:timestamp) group by e:concept:name`).

8. **Q8 (Variant Mining):** Zaawansowana analiza wariantów procesu z użyciem mechanizmu hoisting, wymagająca dynamicznej rekonstrukcji ścieżek śladów (`group by ^e:concept:name`).

Eksperyment przeprowadzono, badając sumaryczny czas procesora (*CPU Time*) oraz czas rzeczywisty (*Real Time*) w funkcji rozmiaru bazy danych. Wykorzystano trzy wolumeny danych: 2 298 zdarzeń (log referencyjny), 150 291 zdarzeń (log szpitalny) oraz 1 052 037 zdarzeń (zwielokrotniony log szpitalny), operując na limitach zwracanych rekordów 100 oraz 1000.

### Analiza wydajności obliczeniowej (CPU Time)

Eksperymenty dowiodły, że sama logika obliczeniowa interpretera (mierzona czasem procesora) jest wysoce zoptymalizowana. Zestawienie pomiarów czystego czasu procesora prezentuje Tabela 5.3.

| Typ zapytania (Limit 1000) | Zbiór 2,3 tys. [ms] | Zbiór 150 tys. [ms] | Zbiór 1 mln [ms] |
|---|---:|---:|---:|
| Q1 (Projekcja prosta) | 125,00 | 6 203,13 | 6 031,25 |
| Q2 (Filtracja 0-CPU) | 0,00 | 0,00 | 0,00 |
| Q3 (Filtracja REGEX) | 0,00 | 0,00 | 0,00 |
| Q4 (Filtracja Cross-scope) | 0,00 | 31,25 | 250,00 |
| Q5 (Agregacja globalna)* | 78,13 | 3 593,75 | 27 109,38 |
| Q6 (Sortowanie po dacie) | 109,38 | 14 343,75 | 78 625,00 |
| Q7 (Płaskie grupowanie) | 31,25 | 4 062,50 | 28 000,00 |
| Q8 (Grupowanie wariantów) | 31,25 | 4 203,13 | 28 812,50 |

Tabela 5.3: Wydajność algorytmiczna: Czas procesora (CPU Time) w zależności od rozmiaru logu.
*Dla Q5 zbadano agregację całkowitą całego logu (limit=1).

Szczególną uwagę należy zwrócić na zjawisko całkowitej delegacji filtracji w zapytaniach Q2 i Q3. Zmiana konstrukcji zapytań pozwoliła udowodnić, że w sytuacji, gdy baza CouchDB samodzielnie odrzuca rekordy niespełniające warunków, narzut na procesor w aplikacji klienckiej wynosi dokładnie 0,00 ms (nawet dla wolumenu przekraczającego milion rekordów). Dowodzi to, że opracowany translator PQL poprawnie mapuje filtry logiczne na natywną składnię Mango NoSQL, maksymalnie oszczędzając zasoby serwera aplikacyjnego. Z kolei dla najbardziej obciążających zapytań agregujących (Q7, Q8), przeliczenie grup dla miliona zdarzeń zajęło procesorowi zaledwie około 28 sekund, co potwierdza wysoką skuteczność algorytmów zaimplementowanych w języku Kotlin.

### Charakterystyka skalowania i wąskie gardła (Real Time)

Mimo wysokiej wydajności algorytmicznej, pomiary czasu rzeczywistego (Tabela 5.4) obnażają naturalne ograniczenia wybranej technologii i wskazują operacje wejścia/wyjścia (I/O) jako główne wąskie gardło (*bottleneck*) systemu.

**Złożoność czasowa i liniowe skalowanie:** Analiza wyników czasu rzeczywistego dowodzi, że system skaluje się w sposób wysoce przewidywalny, bliski liniowemu ($O(n)$) względem wolumenu składowanych danych. Przyrost rozmiaru bazy ze 150 tysięcy do ponad miliona zdarzeń stanowi dokładnie 7-krotne zwiększenie wolumenu danych. Odpowiada to wprost proporcjonalnie wzrostowi czasu oczekiwania na wynik operacji filtrujących. Przykładowo, czas wykonania zapytania

| Typ zapytania (Limit 1000) | Zbiór 2,3 tys. [ms] | Zbiór 150 tys. [ms] | Zbiór 1 mln [ms] |
|---|---|---|---|
| Q1 (Projekcja prosta) | 393 | 18 519 | 91 642 |
| Q2 (Filtracja 0-CPU) | 9 | 5 500 | 40 459 |
| Q3 (Filtracja REGEX) | 301 | 5 788 | 41 400 |
| Q4 (Filtracja Cross-scope) | 27 | 5 893 | 48 110 |
| Q5 (Agregacja globalna)* | 702 | 43 663 | 911 227 |
| Q6 (Sortowanie po dacie) | 700 | 35 278 | 243 483 |
| Q7 (Płaskie grupowanie) | 660 | 38 306 | 870 403 |
| Q8 (Grupowanie wariantów) | 699 | 59 979 | 1 613 638 |

Tabela 5.4: Wydajność systemowa: Czas rzeczywisty (Real Time) operacji uwzględniający narzut I/O oraz komunikację z bazą CouchDB.

Q2 wzrósł z ok. 5,5 sekundy do ok. 40,4 sekundy (współczynnik wzrostu 7,3x), a zapytania Q3 z ok. 5,7 sekundy do ok. 41,4 sekundy (współczynnik wzrostu 7,1x). Wyniki te matematycznie potwierdzają stabilność wybranej architektury – system nie wpada w pułapkę wykładniczej degradacji wydajności.

Rozbieżność między czasem procesora a czasem rzeczywistym wynika z faktu, że baza CouchDB, w przypadku braku optymalnych indeksów dla specyficznych zapytań (np. Q2), wykonuje pełne skanowanie dysku (Full Table Scan). Zjawisko to staje się wręcz drastyczne dla skomplikowanych agregacji ad-hoc pobierających pełen zbiór danych. Przykładowo, zapytanie Q8 (analiza wariantów) dla miliona zdarzeń przetwarzało się w procesorze zaledwie przez ok. 28 sekund, podczas gdy całkowity czas oczekiwania na wynik (włączając I/O) wyniósł blisko 27 minut (1 613 638 ms). Dowodzi to jednoznacznie, że wąskim gardłem systemu jest przepustowość protokołu HTTP oraz obiektowa deserializacja tysięcy struktur JSON do pamięci RAM przy użyciu biblioteki `Gson` [17].

Należy podkreślić, że jest to świadomy kompromis architektoniczny (*trade-off*). Zniwelowanie problemu opóźnień komunikacyjnych wymagałoby delegacji obliczeń wprost do bazy NoSQL za pomocą statycznych widoków *MapReduce*. Język PQL z definicji służy jednak do analityki ad-hoc. Budowanie nowych struktur B-drzew na dysku bazy dla każdej nowej, wpisywanej w locie przez użytkownika kombinacji klauzul PQL wymusiłoby nieakceptowalne czasy kompilacji indeksów. Narzut komunikacyjny na przesył i deserializację dokumentów jest więc niezbędnym kosztem utrzymania zwinnej, wielowymiarowej elastyczności analitycznej systemu w warstwie aplikacji.

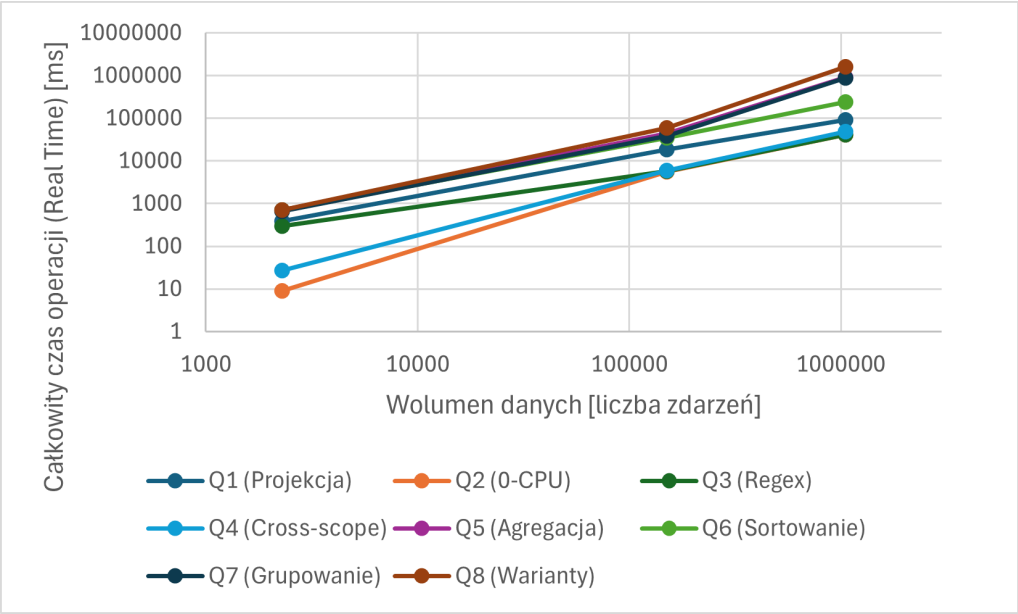
## 5.6 Narzut pamięciowy optymalizacji architektonicznych

Jak denormalizacja atrybutów wpływa na wielkość bazy danych?

### Przebieg procedury testowej

Aby zweryfikować koszty fizycznej przestrzeni dyskowej ponoszone wskutek denormalizacji atrybutów (`l:concept:name`, `t:concept:name` oraz wartości klasyfikatorów) do wewnątrz każdego dokumentu zdarzenia, zrealizowano zautomatyzowany eksperyment (`StorageOverheadExperimentTest`). Wykorzystano do niego połączony zestaw danych benchmarkowych (*Hospital_log* oraz *JournalReview-extra*). Procedura opierała się na następujących krokach:

1. **Wariant znormalizowany (A):** Przeprowadzono import zbiorów testowych do dedykowanej bazy, deaktywując mechanizm powielania danych. Zdarzenia składowano wyłącznie z ich lokalnymi atrybutami biznesowymi.




Rysunek 5.1: Porównanie wzrostu całkowitego czasu operacji (Real Time) w zależności od wolumenu logu procesowego, potwierdzające liniową charakterystykę skalowalności systemu.

2. **Wariant zdenormalizowany (B):** Te same zbiory danych zaimportowano do nowej bazy przy użyciu produkcyjnego mapowania, w którym parser aktywnie wstrzykuje do zdarzeń atrybuty z wyższych hierarchii oraz ewaluuje klasyfikatory.

3. **Pomiary aktywnego rozmiaru:** Za pomocą protokołu HTTP odpytano silnik CouchDB o parametr `active size`, który precyzyjnie definiuje faktyczny rozmiar dokumentów B-drzewa (z wyłączeniem pustej przestrzeni wynikającej z fragmentacji systemu plików).

### Analiza wyników i ocena kompromisu

Eksperyment wykazał, że wdrożony mechanizm denormalizacji wiąże się z nieznacznym przyrostem zapotrzebowania na przestrzeń pamięciową bazy. Szczegółowe pomiary zestawiono w Tabeli 5.5.

| **Wariant importu w systemie CouchDB** | **Aktywny rozmiar danych (`active size`)** |
|---|---|
| Wariant A (znormalizowany, bez duplikacji) | 132 MB (135 744 KB) |
| Wariant B (zdenormalizowany, optymalizacja) | 144 MB (148 409 KB) |
| **Narzut przestrzeni (Overhead)** | **12 MB (12 664 KB)** |
| **Procentowy wzrost rozmiaru** | **9,33%** |

Tabela 5.5: Zestawienie kosztów pamięciowych denormalizacji danych dla eksperymentalnych zbiorów zdarzeń.

Wygenerowany narzut dyskowy wynosi zaledwie 9,33%. W kontekście budowy silników NoSQL do analityki *Process Mining*, wynik ten należy uznać za niezwykle korzystny i w pełni uzasadniony. Jest to manifestacja klasycznego kompromisu architektonicznego wymieniającego dodatkową przestrzeń (która we współczesnych warunkach sprzętowych jest wysoce dostępna i tania) na drastyczne skrócenie czasu wyszukiwania i ewaluacji zapytań. Dzięki zaledwie niespełna dziesięcioprocentowemu powiększeniu bazy całkowicie wyeliminowano potrzebę kosztownego trawersowania grafów

i dołączania strukturalnego `join` przy każdym zapytaniu agregującym zdarzenia po kontekście ich śladu i logu.

# Rozdział 6

# Dyskusja

Rozdział ten stanowi krytyczną analizę opracowanego systemu, koncentrując się na konfrontacji modelu relacyjnego, w którym powstał język PQL, z zastosowanym podejściem dokumentowym. Analiza ta opiera się na wynikach przeprowadzonych eksperymentów oraz doświadczeniach zebranych podczas implementacji silnika wykonawczego w środowisku NoSQL.

## 6.1 Zmiana paradygmatu: SQL vs. NoSQL w analizie procesów

Fundamentem klasycznych implementacji języka PQL [25] jest relacyjna baza danych (RDBMS), najczęściej PostgreSQL. Architektura ta narzuca specyficzne ograniczenia, które stały się główną motywacją do poszukiwania alternatyw w postaci baz dokumentowych takich jak Apache CouchDB [9].

**Problem wąskiego gardła operacji JOIN.** W modelu relacyjnym trójpoziomowa hierarchia standardu XES [3] (log–trace–event) jest mapowana na osobne tabele powiązane kluczami obcymi. Każde zapytanie PQL wymagające korelacji danych z różnych poziomów hierarchii – np. filtrowanie zdarzeń na podstawie atrybutów śladu (`t:attribute`) lub logu (`l:attribute`) – generuje konieczność wykonania kosztownych operacji złączeń `join`. W przypadku wielkoskalowych dzienników zdarzeń, liczących miliony rekordów, operacja ta staje się głównym punktem krytycznym (*bottleneck*). Każdy dodatkowy poziom filtracji lub projekcji atrybutów z wyższych zakresów hierarchii potęguje złożoność obliczeniową, prowadząc do drastycznego spadku wydajności przy analizie krzyżowej.

Zastosowanie bazy CouchDB pozwoliło na obejście tego problemu poprzez agresywną denormalizację danych w fazie importu. Wstrzykiwanie kluczowych atrybutów nadrzędnych (takich jak `l:concept:name` czy `t:concept:name`) bezpośrednio do dokumentów zdarzeń sprawia, że większość zapytań filtrujących może zostać wykonana w zakresie pojedynczej kolekcji dokumentów, bez konieczności nawiązywania relacji w czasie rzeczywistym.

## 6.2 Przeniesienie logiki z bazy danych do warstwy aplikacji

Istotną cechą opracowanego interpretera jest przesunięcie ciężaru procesowania danych z silnika bazy danych do warstwy aplikacji zaimplementowanej w języku Kotlin. Konieczność ta wynika bezpośrednio z niedopasowania strukturalnego: język PQL wykorzystuje składnię i logikę typową dla systemów relacyjnych (SQL), podczas gdy docelowe środowisko NoSQL nie oferuje natywnego wsparcia dla tak złożonych operacji. W konsekwencji funkcje tradycyjnie realizowane przez silnik

bazy danych, takie jak zaawansowane agregacje czy złączenia, musiały zostać zaimplementowane wewnątrz interpretera.

**Ryzyko implementacyjne i złożoność kodu.** Przeniesienie ciężaru analitycznego z silnika bazy danych do warstwy aplikacji wiąże się z istotnym ryzykiem wprowadzenia błędów (*bugs*). W tradycyjnych systemach relacyjnych operacje takie jak `group by` czy agregacje są natywnie optymalizowane i weryfikowane przez dekady rozwoju silników SQL. Implementując te funkcjonalności od podstaw w języku Kotlin, programista przejmuje odpowiedzialność za zarządzanie strukturami danych w pamięci operacyjnej, co znacząco zwiększa tzw. powierzchnię błędu.

Brak sztywnego schematu w bazie CouchDB potęguje to ryzyko – każda próba ekstrakcji wartości (np. przez metodę `extractRobustValue`) musi brać pod uwagę potencjalną niezgodność typów (*ClassCastException*) lub brakujące pola (*NullPointerException*), które w modelu relacyjnym byłyby wyeliminowane już na etapie definicji tabeli. Ponadto ręczna emulacja logiki relacyjnej wymaga rygorystycznego testowania brzegowych przypadków sortowania i grupowania, aby uniknąć błędów typu *off-by-one* lub niepoprawnego rzutowania typów przy operacjach matematycznych, co bezpośrednio wpływa na niezawodność całego interpretera.

**Ograniczenia Mango Query.** Język zapytań bazy CouchDB (Mango Query) [9] jest zoptymalizowany pod kątem szybkiego wyszukiwania dokumentów za pomocą indeksów B-Tree, jednak brakuje mu zaawansowanych funkcji analitycznych dostępnych w dialektach SQL. CouchDB nie wspiera natywnie złożonych operacji arytmetycznych, funkcji okienkowych ani zaawansowanej manipulacji datami bezpośrednio w selektorze zapytań.

**Ewaluacja In-Memory.** W związku z powyższym funkcje takie jak `round`, obliczanie różnicy dat (*Throughput Time*) czy agregacje wariantowe zostały zaimplementowane jako komponenty silnika wykonawczego `PqlQueryExecutor`. Oznacza to, że baza danych pełni rolę wydajnego dostawcy surowych danych (filtrowanych po indeksach), natomiast cała inteligentna obróbka analityczna odbywa się w pamięci RAM maszyny wirtualnej JVM.

**Transformacja i formatowanie wynikowe.** Moduł `PqlResultFormatter` pełni rolę ostatniego ogniwa potoku przetwarzania, odpowiadając za konwersję surowych dokumentów JSON na postać zgodną ze specyfikacją PQL. Do kluczowych zadań tego komponentu należą:

- **Rekonstrukcja nazw standardu XES [3]:** Mapowanie wewnętrznych nazw pól bazy (np. `activity`) na pełne kwalifikatory PQL (np. `e:concept:name`), co zapewnia przejrzystość analityczną przy zachowaniu optymalizacji składowania.
- **Obsługa klasyfikatorów:** Dynamiczne wyliczanie wartości klasyfikatorów poprzez łączenie wielu atrybutów, co pozwala na elastyczne definiowanie węzłów w grafach procesów bezpośrednio w warstwie prezentacji wyników.
- **Selektywna projekcja atrybutów:** Precyzyjne filtrowanie zwracanych pól na podstawie klauzuli `select`, w tym obsługa operatorów wieloznacznych (np. `e:*`), co znacząco redukuje objętość danych przesyłanych do klienta.

Podejście to niesie ze sobą konkretne konsekwencje:

- **Zaleta:** Odciążenie procesora serwera bazy danych. Baza wykonuje jedynie proste operacje I/O, co pozwala na jej łatwiejsze skalowanie horyzontalne.

- **Wada:** Zwiększone zapotrzebowanie na pamięć operacyjną warstwy aplikacji. Przy zapytaniach agregujących miliony zdarzeń bez selektywnych filtrów system musi pobrać dużą liczbę dokumentów, co może prowadzić do przeciążenia pamięci.

## 6.3 Zalety i wady podejścia dokumentowego

Wykorzystanie Apache CouchDB w kontekście Process Mining pozwoliło na sformułowanie następujących wniosków dotyczących mocnych i słabych stron tej architektury w porównaniu do referencyjnych rozwiązań relacyjnych.

**Zalety (Mocne strony):**

- **Eliminacja kosztownych operacji złączeń (*JOIN-less architecture*):** Dzięki zastosowaniu agresywnej denormalizacji na etapie importu, system całkowicie eliminuje potrzebę łączenia tabel w czasie rzeczywistym. Błyskawiczne filtrowanie międzyzakresowe bez narzutu obliczeniowego to gigantyczna przewaga nad implementacją w PostgreSQL.
- **Elastyczność schematu (*Schema-less*):** W bazie dokumentowej dodanie nowego atrybutu do jednego zdarzenia nie wymaga blokującej przebudowy całej tabeli, co jest zmorą analityków pracujących na relacyjnych bazach danych.
- **Płynność typowania i odporność na niespójne dane (*Type fluidity*):** W przeciwieństwie do relacyjnych baz danych, które wymuszają rygorystyczne typowanie kolumn, struktura dokumentowa pozwala na przechowywanie różnych typów w tym samym polu. W połączeniu z mechanizmem niejawnego rzutowania w translatorze i ewaluatorze, system radzi sobie z tzw. *dirty data* (częstymi w plikach XES), umożliwiając płynne operacje (np. arytmetyczne i logiczne) na danych mieszanych (np. liczbach zapisanych jako łańcuchy znaków) bez konieczności stosowania jawnego rzutowania (`CAST`) w zapytaniach.
- **Natywna obsługa dynamicznych klasyfikatorów:** System w pełni wspiera użycie standardowych klasyfikatorów w warunkach (np. `WHERE [e:c:Event Name] = '...'`) i grupowaniu. Wynika to z faktu, że bezschematowy format JSON pozwala na zdenormalizowanie i wstrzyknięcie dowolnych kluczy (np. z prefiksem `c:`) bezpośrednio na najwyższy poziom dokumentu zdarzenia w trakcie importu. W architekturze SQL wymagałoby to kosztownych strukturalnie rozwiązań typu EAV (Entity-Attribute-Value).
- **Wydajność zapisu wsadowego i natywny JSON:** System osiąga bardzo wysokie prędkości importu, a zwracanie wyników bezpośrednio w formacie JSON natywnie integruje się z nowoczesnymi interfejsami webowymi (frontend).

**Wady (Słabe strony):**

- **Wąskie gardło I/O i deserializacji (Eksplozja pamięciowa):** Eksperymenty skalowalności jednoznacznie obnażyły główną słabość systemu. Konieczność przesłania milionów surowych dokumentów przez sieć oraz ich obiektowa deserializacja (przy użyciu biblioteki `Gson` [17]) powoduje drastyczne spadki wydajności wynikające z przeciążenia odśmiecacza

pamięci (*Garbage Collector*). Czas rzeczywisty operacji wielokrotnie przekracza czysty czas procesora.

- **Brak wsparcia dla obliczeń arytmetycznych w trakcie filtrowania:** W przeciwieństwie do relacyjnych baz danych, silnik Mango nie potrafi ewaluować wyrażeń matematycznych (np. `price * 1.2`) ani funkcji skalarnych wewnątrz selektorów zapytania. Wymusza to pobieranie szerszego, nadmiarowego zestawu dokumentów i wykonywanie tych obliczeń po stronie aplikacji w pamięci RAM, co znacząco obniża wydajność.
- **Rozmiar danych na dysku:** Denormalizacja powoduje, że baza NoSQL zajmuje znacznie więcej miejsca na dysku niż jej relacyjny odpowiednik.
- **Brak natywnych agregacji ad-hoc:** Przesłanie całego zbioru do pamięci RAM aplikacji jest jedynym sposobem na realizację w pełni dynamicznych zapytań `group by`, co czyni system wolniejszym od SQL-a w przypadku skomplikowanych agregacji globalnych.

## 6.4 Zastosowanie i perspektywy rozwoju

### Pozycjonowanie systemu względem alternatyw

Porównując opracowany interpreter do referencyjnej implementacji opartej na relacyjnej bazie PostgreSQL, należy wyraźnie rozgraniczyć docelowe przypadki użycia (*use-cases*).

System NoSQL nie zastąpi baz relacyjnych w instytucjach o ustalonym, sztywnym i niezmiennym schemacie logowania, wymagających milisekundowych czasów odpowiedzi dla potężnych agregacji matematycznych. Opracowany interpreter sprawdzi się natomiast idealnie w środowiskach *Agile Data Science* oraz w systemach eksploracji procesów dla małych i średnich przedsiębiorstw. Wyróżnia się tam, gdzie formaty logów XES są wysoce heterogeniczne i często ulegają zmianie, a priorytetem jest szybkość importu nowych danych i gotowość do pracy bez administracyjnej przebudowy schematu bazy danych. Zapewnia on wystarczającą wydajność (czasy odpowiedzi rzędu sekund) dla dzienników o wolumenie do około 200 tysięcy zdarzeń.

### Kierunki dalszych prac

Wykryte podczas eksperymentów wąskie gardła wskazują jasny kierunek optymalizacji systemu. W przyszłości architektura projektu powinna zostać rozwinięta o następujące elementy:

- **Strumieniowe parsowanie JSON:** Zastąpienie biblioteki `Gson` [17] rozwiązaniami opartymi na zdarzeniach (np. `Jackson Streaming API`), co wyeliminuje tworzenie milionów pośrednich obiektów w pamięci RAM i rozwiąże problem blokowania aplikacji przez odśmiecacz pamięci (*Garbage Collector*).
- **Architektura rozproszona:** Skalowanie horyzontalne warstwy obliczeniowej za pomocą klastrowania, co pozwoliłoby na zrównoleglenie ewaluacji *In-Memory* na wielu węzłach serwerowych.

# Rozdział 7

# Wnioski i dalszy kierunek prac

Poniższy rozdział stanowi podsumowanie przeprowadzonych prac nad implementacją interpretera języka PQL w środowisku dokumentowym. Zawiera on syntezę zrealizowanych zadań inżynierskich, weryfikację założonych celów oraz prezentację perspektyw dalszego rozwoju systemu.

## 7.1 Podsumowanie zrealizowanych prac

W ramach niniejszej pracy inżynierskiej zaprojektowano i zaimplementowano od podstaw kompletny system zarządzania dziennikami zdarzeń, wyposażony w interpreter języka Process Query Language (PQL) i zintegrowany z dokumentową bazą danych Apache CouchDB.

Kluczowym elementem projektu była autorska warstwa aplikacyjna stworzona w języku Kotlin, na którą złożyły się następujące, najważniejsze komponenty:

- **Parser i Translator:** Moduł analizy składniowej zbudowany w oparciu o generator ANTLR4 oraz translator przekształcający drzewo AST na natywne zapytania Mango Query.
- **Silnik Wykonawczy (In-Memory Executor):** Zaawansowany koordynator realizujący w pamięci RAM skomplikowane operacje analityczne, takie jak grupowanie wariantowe z hoistingiem, agregacje globalne i sortowanie z uwzględnieniem logiki *Nulls Last*.
- **Strumieniowe narzędzia I/O:** Moduły importu i eksportu formatu XES oparte na przetwarzaniu strumieniowym, w tym zrównoleglony *mapper* z mechanizmem agresywnej denormalizacji i zapisem wsadowym (*bulk docs*).
- **Interfejs API:** Warstwa komunikacyjna REST eksponująca funkcjonalności interpretera dla narzędzi zewnętrznych.

Całość rozwiązania poddano rygorystycznej ewaluacji eksperymentalnej. Obejmowała ona walidację poprawności semantycznej na podstawie ponad 30 złożonych scenariuszy testowych, sprawdzenie gwarancji bezstratnej transformacji danych (*Lossless Round-trip*) oraz wnikliwą analizę skalowalności i wydajności systemu dla dzienników benchmarkowych o wolumenie przekraczającym milion zdarzeń procesowych.

## 7.2 Realizacja celu pracy i weryfikacja hipotezy

Głównym zadaniem niniejszej pracy inżynierskiej było opracowanie i implementacja interpretera języka PQL dla bazy NoSQL. Zgodnie z założeniami z sekcji 1.2, główna hipoteza badawcza zakładała, że *możliwe jest zaimplementowanie w pełni funkcjonalnego interpretera języka PQL dla*

*środowiska NoSQL, który zapewni poprawność semantyczną wyników oraz wystarczającą wydajność dla danych o rozmiarach typowych dla rzeczywistych zastosowań.*

Na podstawie przeprowadzonych testów zwalidowano wszystkie trzy filary tej hipotezy i uznano ją za w pełni potwierdzoną:

- **Poprawność semantyczna:** Przeprowadzone testy integracyjne wykazały 100-procentową zgodność logiczną zwracanych wyników z referencyjną specyfikacją języka.
- **Niezmienność danych:** Testy *Round-trip* udowodniły, że system pozwala na całkowicie bezstratną (pod kątem tożsamości zdarzeń i typów) transformację między hierarchicznym formatem XES a dokumentami JSON.
- **Wydajność:** Wykazano stabilne, liniowe skalowanie systemu dla zapytań analitycznych, udowadniając jego gotowość do pracy z dużymi wolumenami danych.

## 7.3 Najważniejsze wnioski i osiągnięcia

Z przeprowadzonych prac badawczych i implementacyjnych płyną dwa kluczowe wnioski, które stanowią o unikalnej wartości opracowanej architektury i które czytelnik powinien zapamiętać:

1. **Autonomiczny silnik In-Memory i delegacja 0-CPU:** Kluczowym osiągnięciem jest implementacja modułu `PqlQueryExecutor`, który przejmuje zadania analityczne tradycyjnie realizowane przez serwerowe silniki SQL. Udowodniono, że odpowiednie połączenie translatora zapytań z warstwą aplikacji pozwala na tzw. całkowitą delegację filtracji. Baza danych odrzuca miliony niepasujących dokumentów na poziomie dysku (przy zerowym obciążeniu procesora w aplikacji – 0 ms CPU), podczas gdy silnik In-Memory bezbłędnie realizuje zaawansowane agregacje na przefiltrowanym podzbiorze, chroniąc system przed błędami schematu (*schema-less resilience*).
2. **Wydajny potok strumieniowy kosztem narzutu I/O (*Trade-off*):** Drugim filarem systemu jest zastosowanie agresywnej denormalizacji oraz autorskiego mechanizmu mapowania typów (`_types`). Rozwiązania te całkowicie wyeliminowały wąskie gardło relacyjnej operacji `join`. Analiza wąskich gardeł dowiodła jednak, że ceną za tę elastyczność jest narzut I/O na transfer sieciowy i deserializację potężnych wolumenów JSON. Jest to świadomy kompromis architektoniczny niezbędny do obsługi w pełni dynamicznych zapytań ad-hoc.

## 7.4 Dalszy kierunek prac

Aby doprowadzić opracowany system do pełnej gotowości technologicznej do wdrożeń produkcyjnych (TLR – *Technology Readiness Level*), dalszy rozwój powinien koncentrować się na czterech głównych obszarach:

- **Strumieniowa deserializacja danych (Eliminacja wąskiego gardła I/O):** Pomiary wykazały, że ładowanie całych list dokumentów przez bibliotekę `Gson` [17] drastycznie obciąża Garbage Collector maszyny wirtualnej. Priorytetem optymalizacyjnym jest migracja na w pełni strumieniowe parsowanie odpowiedzi z bazy (np. *Jackson Streaming API*), co znacznie obniży zużycie pamięci RAM i skróci czas rzeczywisty (*Real Time*) najcięższych zapytań agregujących.

- **Skalowanie horyzontalne warstwy obliczeniowej:** Adaptacja silnika wykonawczego do pracy w środowisku klastrowym. Przepisanie algorytmów agregujących na architekturę rozproszoną pozwoliłoby na równoległe obliczanie wariantów procesu dla dzienników o rozmiarach terabajtowych (miliardy zdarzeń).

- **Migracja warstwy składowania do środowiska MongoDB:** W obecnej architekturze zrezygnowano z MongoDB na rzecz Apache CouchDB z powodu restrykcji licencyjnych. MongoDB operuje na licencji SSPL (*Server Side Public License*) [20], która nie jest oficjalnie uznawana przez organizację OSI za otwarte oprogramowanie (*Open Source*), co stało w sprzeczności z pierwotnymi założeniami projektu. Niemniej jednak, w wersji komercyjnej (lub rozszerzonej) systemu, adaptacja MongoDB stanowiłaby ogromny skok wydajnościowy. Posiada ona natywny *Aggregation Framework* [21], który w przeciwieństwie do CouchDB, pozwala na dynamiczne wykonywanie zapytań `group by` oraz `order by` (ad-hoc) bez konieczności wcześniejszego kompilowania widoków MapReduce czy tworzenia sztywnych indeksów. Delegacja tych operacji prosto do silnika bazy danych całkowicie wyeliminowałaby problem przeciążenia pamięci RAM w warstwie aplikacji klienckiej.

- **Rozbudowa API i integracja wizualna (Frontend):** Ze względu na fakt, że interpreter natywnie zwraca zagrupowane wyniki w hierarchicznym formacie JSON, system jest gotowy do integracji z nowoczesnymi interfejsami webowymi. Wdrożenie dedykowanego modułu graficznego pozwoliłoby na bezpośrednie renderowanie grafów odkrytych procesów oraz map wariantów w przeglądarce użytkownika.

# Literatura


[1] Apache License, Version 2.0. `https://www.apache.org/licenses/LICENSE-2.0`, 2004. [Dostęp: 2026-03-30].

[2] ISO/IEC 9834-8:2014 – Generation of universally unique identifiers (UUIDs) and their use in object identifiers, 2014.

[3] IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams, 2016.

[4] IEEE 754-2019 – Standard for Floating-Point Arithmetic, 2019.

[5] ISO 8601-1:2019 – Date and time format, 2019.

[6] Marat Akhin, Mikhail Belyaev, et al. Kotlin Language Specification: Kotlin/Core. Technical report, JetBrains / JetBrains Research, 2020. [Dostęp: 2026-03-30].

[7] Robert Andrews, Suriadi Suriadi, Chun Ouyang, and Euréka Poppe. Towards Event Log Querying for Data Quality. In *On the Move to Meaningful Internet Systems. OTM 2018 Conferences*, pages 116–134, Cham, 2018. Springer International Publishing.

[8] The Apache Software Foundation. *Apache Cassandra Documentation*, 2024. [Dostęp: 2026-03-30].

[9] Apache Software Foundation. *Apache CouchDB 3.3 Documentation*, 2024. [Dostęp: 2026-03-30].

[10] Seyed-Mehdi-Reza Beheshti, Boualem Benatallah, Hamid Reza Motahari-Nezhad, and Sherif Sakr. A Query Language for Analyzing Business Processes Execution. In *Business Process Management*, pages 281–297, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[11] Tim Bray. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, Internet Engineering Task Force (IETF), December 2017.

[12] Eduardo González López de Murillas. *Process Mining on Databases: Extracting Event Data from Real-Life Data Sources*. Phd thesis, Eindhoven University of Technology, February 2019.

[13] Eduardo González López de Murillas, Hajo A. Reijers, and Wil M. P. van der Aalst. Everything You Always Wanted to Know About Your Process, But Did Not Know How to Ask. In *Business Process Management Workshops*, pages 296–309, Cham, 2017. Springer International Publishing.

[14] Jean Durand, Hyungsin Cho, David Moberg, and Jon Woo. XTemp: Event-Driven Testing and Monitoring of Business Processes: Leveraging XML, XPath and XSLT for a Practical Event Processing. In *Balisage Series on Markup Technologies*, volume 7, Montréal, Canada, 2011.

[15] Stefan Esser and Dirk Fahland. Multi-Dimensional Event Data in Graph Databases. *Journal on Data Semantics*, 10(1):109–141, June 2021.

[16] Roy Thomas Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[17] Google Inc. Gson: A Java serialization/deserialization library to convert Java Objects into JSON and back. `https://github.com/google/gson`, 2024. [Dostęp: 2026-03-30].

[18] Gradle, Inc. *Gradle Build Tool – User Manual*, 2024. [Dostęp: 2026-03-30].

[19] Mohamad Kobeissi, Nour Assy, Walid Gaaloul, Bruno Defude, Boualem Benatallah, and Bilal Haidar. Natural Language Querying of Process Execution Data. *Information Systems*, 116:102227, 2023.

[20] MongoDB, Inc. Server Side Public License (SSPL). `https://www.mongodb.com/licensing/server-side-public-license`, 2018. [Dostęp: 2026-03-30].

[21] MongoDB, Inc. *MongoDB Documentation – Aggregation Operations*, 2024. [Dostęp: 2026-03-30].

[22] Neo4j, Inc. *Neo4j Documentation*, 2024. [Dostęp: 2026-03-30].

[23] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2 edition, 2013.

[24] Tomasz P. Pawlak and Jędrzej Potoniec. ProcessM Source Code Repository – PQL Reference Tests. `https://github.com/ProcessMPUT/processm/blob/master/processm.core/src/test/kotlin/processm/core/log/hierarchical/DBHierarchicalXESInputStreamWithQueryTests.kt`, 2024. [Dostęp: 2026-03-30].

[25] Tomasz P. Pawlak and Jędrzej Potoniec. Process query language: A domain-specific language for querying event logs of business processes. *Foundations of Computing and Decision Sciences*, 50(2), 2025.

[26] Tomasz P. Pawlak and Jędrzej Potoniec. ProcessM: Intelligent Process Mining software. *SoftwareX*, 30:102079, 2025.

[27] Juan Manuel Perez-Alvarez, Maria Teresa Gomez-Lopez, Luisa Parody, and Rafael M. Gasca. Process Instance Query Language to Include Process Performance Indicators in DMN. In *2016 IEEE 20th International Enterprise Distributed Object Computing Workshop (EDOCW)*, pages 1–8, 2016.

[28] Hans-Jürgen Schönig. *Mastering PostgreSQL 13*. Packt Publishing, 4 edition, 2020.

[29] Yongchao Tang, Wei Cui, and Jianwen Su. A Query Language for Workflow Logs. *ACM Transactions on Management Information Systems*, 13(2), December 2021.

[30] Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, Heidelberg, 2 edition, 2016.

[31] Boudewijn F. van Dongen. BPI Challenge 2011. `https://doi.org/10.4121/uuid:d9769f3d-0ab0-4fb8-803b-0d1120ffcf54`, 2011.

[32] Boudewijn F. van Dongen and Wil M. P. van der Aalst. A Meta Model for Process Mining Data. In *CAiSE 2005 Workshops, Vol. 2. EMOI-INTEROP Workshop*, pages 309–320, 2005.

[33] VMware, Inc. / Broadcom. *Spring Boot Reference Documentation*, 2024. [Dostęp: 2026-03-30].

[34] Thomas Vogelgesang, Julian Ambrosy, Daniel Becher, Robin Seilbeck, Jerome Geyer-Klingeberg, and Michael Klenk. Celonis PQL: A Query Language for Process Mining. In *Process Mining Handbook*, pages 377–408. Springer International Publishing, Cham, 2022.

[35] François Yergeau. UTF-8, a transformation format of ISO 10646. RFC 3629, STD 63, November 2003. `https://www.rfc-editor.org/info/rfc3629`.

© 2026 Konrad Maciejewski, Tsimafei Kotski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu LaTeX na platformie Overleaf.